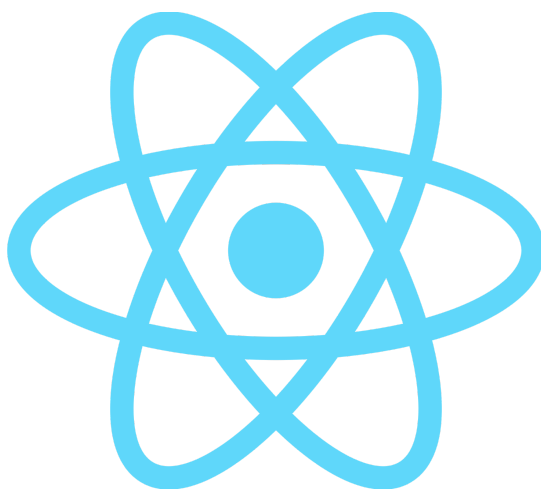


Učebnice React pro středně pokročilé

k videím na YT Hackni svou budoucnost

David Šetek



Odkaz na učebnici

<https://bit.ly/hackni-react-2>

Seznam videí na kanále YouTube

David Šetek - Hackni svou budoucnost

https://www.youtube.com/playlist?list=PLQ8x_VWW6Aktmp_v1eYcCBbO5GovP_sHI

1. React 2 - Vytvoření formuláře

Video: <https://youtu.be/NYf9hNqADYs>

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result"></div>
</body>
</html>
```

index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

App.js

Formulář musí být vždy mezi tagy form. Políčka vytváříme pomocí tagu input. Musíme zadat typ inputu (políčka). Zde jsme zvolili typ text (pro zadání jména) a poté typ submit (pro odeslání).

```
const App = () => {
  return <article>
    <form>
      <input type="text" id="userName" placeholder="Jméno" />
      <input type="submit" />
    </form>
  </article>
}

export default App
```

Document

localhost:30...

Jméno

Odeslat

2. React 2 - Stylování formuláře

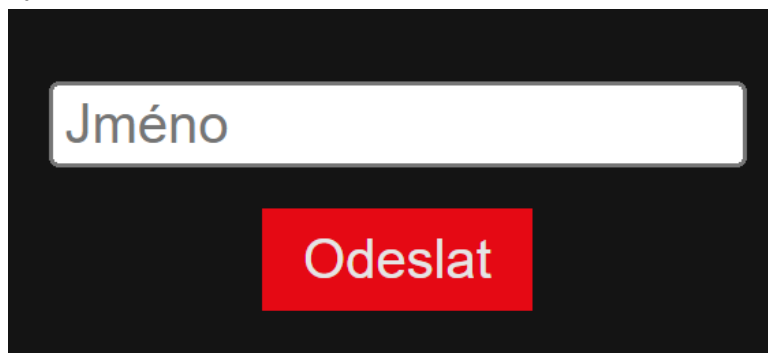
Video: <https://youtu.be/Yglu54eSMRo>

Index.css

Provedeme základní stylování.

```
body {  
  margin: 0;  
  padding: 0;  
  background-color: #141414;  
  color: #e5e5e5;  
}  
  
form {  
  height: 100vh;  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
}  
  
.userName {  
  margin-bottom: 10px;  
  outline: none;  
}  
  
input[type="submit"] {  
  background-color: #e50914;  
  color: #e5e5e5;  
  border: none;  
  padding: 5px 10px;  
}
```

Výsledek na stránce



Jméno

Odeslat

3. React 2 - zachytáváme odeslání formuláře (onSubmit, onClick)

Video: <https://youtu.be/mjw1AsQgxEM>

App.js

První možnost zachycení toho, že došlo k odeslání formuláře. U funkce formSubmit přidáme slovo event do závorek a spustíme preventDefault(), abychom se vyhnuli výchozímu znovunačtení stránky po kliknutí na tlačítko Odeslat.

```
const App = () => {

  const formSubmit = (event) => {
    event.preventDefault()
    console.log("Formulář byl odeslán!!");
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input className="userName" type="text" placeholder="Jméno" />
      <input type="submit" />
    </form>
  </article>
}

export default App
```

Druhá možnost, jak zachytit odeslání formuláře.

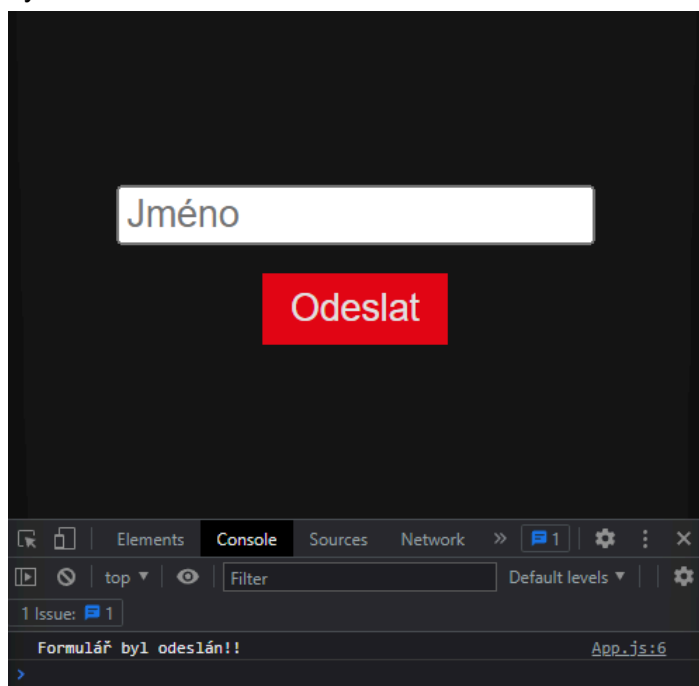
```
const App = () => {

  const formSubmit = (event) => {
    event.preventDefault()
    console.log("Formulář byl odeslán!!");
  }

  return <article>
    <form>
      <input className="userName" type="text" placeholder="Jméno" />
      <input onClick={formSubmit} type="submit" />
    </form>
  </article>
}

export default App
```

Výsledek na stránce



4. React 2 - useState a formulář

Video: <https://youtu.be/6G1jqMUqC08>

App.js

Naimportujeme useState. Pomocí useState nastavíme proměnnou s názvem firstName na hodnotu prázdného stringu. Tuto proměnnou dáme jako hodnotu do value. Takže do políčka se vyplní prázdný string.

```
import { useState } from "react"

const App = () => {
  const [firstName, setFirstName] = useState("")

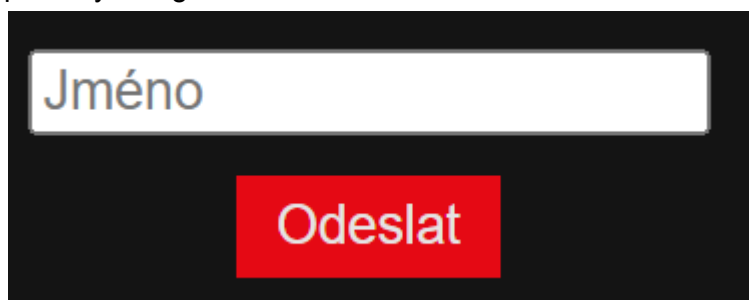
  const formSubmit = (event) => {
    event.preventDefault()
    console.log("Formulář byl odeslán!!");
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input
        className="userName"
        type="text"
        placeholder="Jméno"
        value={firstName}
      />

      <input type="submit" />
    </form>
  </article>
}

export default App
```

Výsledek na stránce - pozor, že do políčka nyní nemůžete psát, protože je tam neustále prázdný string.



The image shows a user interface on a dark background. At the top, there is a white text input field with the placeholder text "Jméno" in a light blue-grey font. Below the input field, centered, is a red rectangular button with the white text "Odeslat".

5. React 2 - ukládáme hodnotu z formuláře

Video: <https://youtu.be/-hDaROLnyr4>

App.js

Jakmile dojde k jakékoli změně v políčku, tak se spustí funkce, která je uvedena za onChange. Načte se obsah políčka (event.target.value) a ten se přes setFirstName uloží do proměnné firstName. Díky useState se celá komponenta znovu vypíše do stránky.

```
import { useState } from "react"

const App = () => {
  const [firstName, setFirstName] = useState("")

  const formSubmit = (event) => {
    event.preventDefault()
    console.log(firstName);
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input
        className="userName"
        type="text"
        placeholder="Jméno"
        value={firstName}
        onChange={(event) => setFirstName(event.target.value)}
      />

      <input type="submit" />
    </form>
  </article>
}

export default App
```

Harry

Odeslat

Elements

Console

Sources

Network

Performance

>>

top ▼

Filter

Default levels ▼

Harry

6. React 2 - vypisujeme hodnotu z formuláře do stránky (špatný postup)

Video: <https://youtu.be/GI00M5ptBWg>

App.js

Vytvoříme si prázdné pole s názvem names. Pokud dojde k odeslání formuláře, tak do names přidáme (push) obsah firstName, kde se nachází obsah políčka formuláře. Dole už jen pomocí funkce map vypíšeme obsah names do stránky.

```
import { useState } from "react"

const names = []

const App = () => {
  const [firstName, setFirstName] = useState("")

  const formSubmit = (event) => {
    event.preventDefault()

    if (firstName){
      names.push(firstName)
      console.log(names)
    } else {
      console.log("Nebylo nic vyplněno");
    }
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input
        className="userName"
        type="text"
        placeholder="Jméno"
        value={firstName}
        onChange={ (event) => setFirstName(event.target.value) }
      />

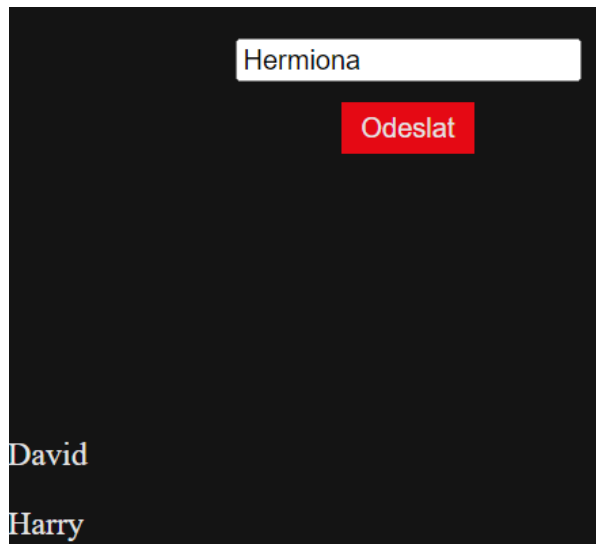
      <input type="submit" />
    </form>

    {names.map( (oneName) => {
      return <p>{oneName}</p>
    })}
```

```
</article>
}

export default App
```

Obsah políčka se vypisuje do stránky, ale vždy je výpis o jedno jméno pozadu. Nefunguje to tak, jak bychom chtěli.



The screenshot shows a web interface with a dark background. At the top, there is a white text input field containing the name 'Hermiona'. Below the input field is a red button with the text 'Odeslat' in white. At the bottom of the visible area, there is a list of names: 'David' and 'Harry'.

7. React 2 - vypisujeme hodnotu z formuláře do stránky (správný postup)

Video: <https://youtu.be/qjGzq4Zfn3o>

App.js

Vytvoříme proměnnou names pomocí useState a nastavíme ji na prázdné pole. Pokud je firstName vyplněné v pomínce, tak použijeme setNames pro to, abychom obsah firstName přidali k existujícímu names. K tomu použijeme tzv. spread operator (operátor s třemi tečkami). Ještě se k němu v následujících videích vrátíme. Dole ještě bude zapotřebí při výpisu zadat unikátní identifikátor. K tomu využijeme index jmen z pole (0, 1, 2 atd.)

```
import { useState } from "react"

const App = () => {
  const [firstName, setFirstName] = useState("")
  const [names, setNames] = useState([])

  const formSubmit = (event) => {
    event.preventDefault()

    if (firstName) {
      setNames( (names) => {
        return [...names, firstName]
      })
    } else {
      console.log("Nebylo nic vyplněno");
    }
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input
        className="userName"
        type="text"
        placeholder="Jméno"
        value={firstName}
        onChange={(event) => setFirstName(event.target.value)}
      />

      <input type="submit" />
    </form>

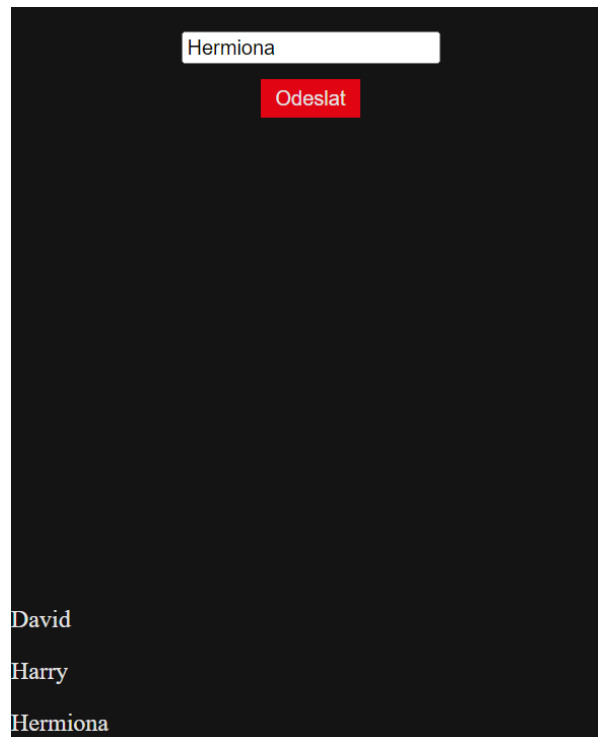
    {names.map( (oneName, index) => {
      return <p key={index}>{oneName}</p>
    })}
```

```
    }} }

    </article>
  }

export default App
```

Výsledek v prohlížeči. Již vše funguje tak, jak má. Až na nastýlování, které opravíme dále.



The screenshot shows a web application with a dark background. At the top, there is a white input field containing the text "Hermiona". Below the input field is a red button with the white text "Odeslat". Below the button, there is a list of names: "David", "Harry", and "Hermiona".

8. React 2 - dostylování aplikace a vyřešení vymazání obsahu políčka po odeslání

Video: <https://youtu.be/msrHyFhd6oA>

App.js

Přidáme vymazání políčka po odeslání formuláře pomocí setFirstName.

Kvůli stylování přidáme každému p classu s názvem item.

```
const App = () => {
  const [firstName, setFirstName] = useState("")
  const [names, setNames] = useState([])

  const formSubmit = (event) => {
    event.preventDefault()

    if (firstName){
      setNames( (names) => {
        return [...names, firstName]
      })
    } else {
      console.log("Nebylo nic vyplněno");
    }

    setFirstName("")
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input
        className="userName"
        type="text"
        placeholder="Jméno"
        value={firstName}
        onChange={(event) => setFirstName(event.target.value)}
      />

      <input type="submit" />
    </form>

    {names.map( (oneName, index) => {
      return <p className="item" key={index}>{oneName}</p>
    })}

  </article>
```

```
}
```

index.css

```
form {  
    height: 30vh;  
    display: flex;  
    flex-direction: column;  
    justify-content: center;  
    align-items: center;  
}  
  
.userName {  
    margin-bottom: 10px;  
    outline: none;  
}  
  
input[type="submit"] {  
    background-color: #e50914;  
    color: #e5e5e5;  
    border: none;  
    padding: 5px 10px;  
    cursor: pointer;  
}  
  
.item {  
    text-align: center;  
    box-shadow: 0px 0px 10px #e50914;  
    width: 300px;  
    margin: 10px auto;  
}
```

Výsledek na stránce

Odeslat

David

Hermiona

Ron

9. React 2 - Spread operátor (spread operator)

Video: <https://youtu.be/qLoxy1M2W4>

Do jakéhokoliv souboru.

```
// Spread operator

// Spojování polí
const studentsGroup = ["David", "Jana", "Martina"]
const allStudents = ["Harry", "Ron", ...studentsGroup, "Hermiona"]
console.log(allStudents)
// Výsledek: ['Harry', 'Ron', 'David', 'Jana', 'Martina', 'Hermiona']

// argumenty funkcí
const numbers = [1, 5, 3]

const multipleNumbers = (a, b, c) => {
  console.log(a * b * c)
}
multipleNumbers(...numbers)
// Výsledek: 15

// Kopírování polí
const students1 = ["Jan", "Monika", "Dalibor"]
const students2 = students1
students2.push("Pavel")
console.log(students2)
console.log(students1)
// Výsledek: ['Jan', 'Monika', 'Dalibor', 'Pavel']
// Výsledek: ['Jan', 'Monika', 'Dalibor', 'Pavel']

const students3 = ["Jan", "Monika", "Dalibor"]
const students4 = [...students3]
students4.push("Pavel")
console.log(students3)
console.log(students4)
// Výsledek: ['Jan', 'Monika', 'Dalibor']
// Výsledek: ['Jan', 'Monika', 'Dalibor', 'Pavel']

// další spojování
const names1 = ["David", "Martin"]
```

```
const names2 = ["Jana", "Pavla"]
const newArray1 = names1.concat(names2)
console.log(newArray1)
const newArray2 = [...names1, "Dana", ...names2]
console.log(newArray2)
// Výsledek: ['David', 'Martin', 'Jana', 'Pavla']
// Výsledek: ['David', 'Martin', 'Dana', 'Jana', 'Pavla']
```

10. React 2 - Rest operátor (rest operator)

Video: <https://youtu.be/B97fzZwqSHU>

Do jakéhokoliv souboru.

```
// Rest operátor
const sum = (...allNumbers) => {
  let result = 0
  for (const oneNumber of allNumbers){
    // result = result + oneNumber
    result += oneNumber
  }
  return result
}

console.log(sum(1, 2))
console.log(sum(1, 2, 3))
console.log(sum(1, 2, 5, 10))
console.log(sum(1, 2, 10, 10, 10))
```

11. React 2 - Pokročilejší formulář - html struktura

Video: https://youtu.be/bmmzl_4ams0

App.js

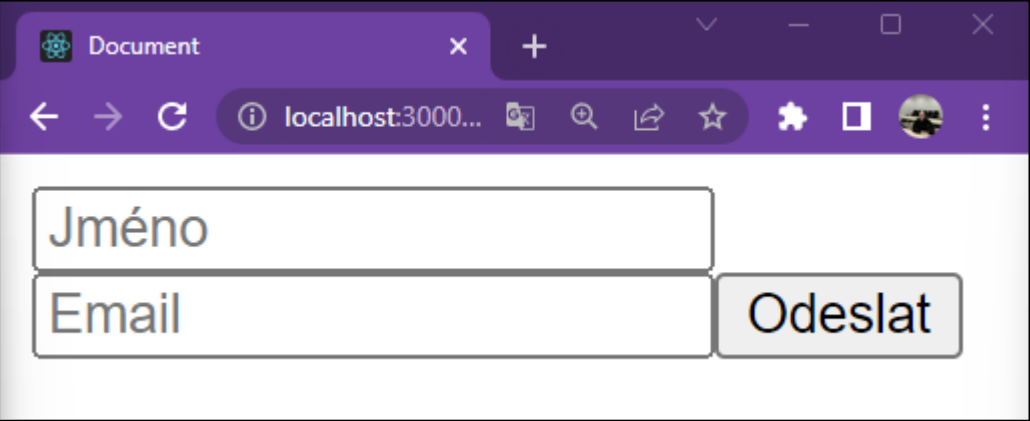
Nejdříve si vytvoříme formulář. Bude mít dvě políčka (jméno a email) a odesílací tlačítko.

```
import React from 'react'

const App = () => {
  return <article>
    <form>
      <input className="userInfo" type="text" placeholder="Jméno" />
      <input className="userInfo" type="email" placeholder="Email" />
      <input type="submit" />
    </form>
  </article>
}

export default App
```

Výsledek na stránce



The screenshot shows a web browser window with a single tab titled 'Document'. The address bar displays 'localhost:3000...'. The page content consists of a form with two input fields stacked vertically. The first input field has the placeholder text 'Jméno' and the second has 'Email'. To the right of the 'Email' field is a button labeled 'Odeslat'.

12. React 2 - Pokročilejší formulář - stylujeme formulář

Video: <https://youtu.be/40pRB4zTRz4>

Index.css

Provedeme stylování formuláře. Classu item nikde použitou nemáme. Připravili jsme si ji dopředu. Použijeme ji, až budeme vypisovat do stránky informace z formuláře.

```
body {
  margin: 0;
  padding: 0;
  background-color: #141414;
  color: #e5e5e5;
}

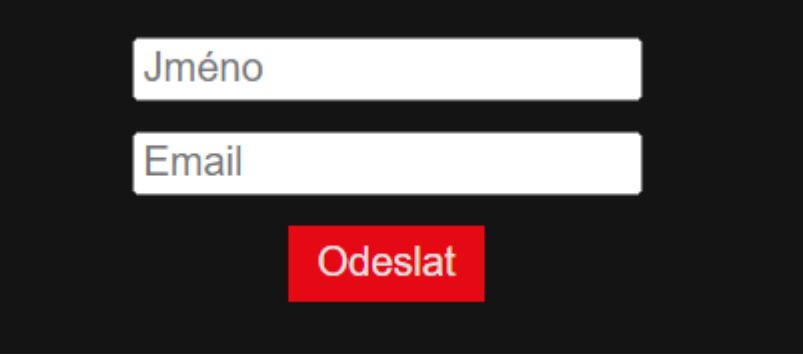
form {
  height: 50vh;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}

.userInfo {
  margin-bottom: 10px;
  outline: none;
}

input[type="submit"] {
  background-color: #e50914;
  color: #e5e5e5;
  border: none;
  padding: 5px 10px;
  cursor: pointer;
}

.item {
  text-align: center;
  box-shadow: 0px 0px 10px #e50914;
  width: 300px;
  margin: 10px auto;
  padding-top: 5px;
  padding-bottom: 5px;
}
```

Výsledek na stránce



A dark-themed login form with two input fields labeled "Jméno" and "Email", and a red "Odeslat" button.

Jméno

Email

Odeslat

13. React 2 - Pokročilejší formulář - odeslání formuláře

Video: https://youtu.be/pyKyDtTCE_U

App.js

Naimportujeme useState a pomocí useState vytvoříme proměnné, které představují obě políčka (fullName a email). Poté ještě vytvoříme proměnnou users, kam budeme ukládat informace z formuláře po odeslání. Users jsou nyní prázdným polem, které postupně budeme plnit.

Přidáme zachycení odeslání formuláře, které spustí funkci formSubmit. Vypneme znovunačtení stránky po odeslání pomocí preventDefault().

```
import { useState } from "react"

const App = () => {
  const [fullName, setFullName] = useState("")
  const [email, setEmail] = useState("")
  const [users, setUsers] = useState([])

  const formSubmit = (event) => {
    event.preventDefault()
    console.log("Formulář byl odeslán!!")
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input className="userInfo" type="text" placeholder="Jméno" />
      <input className="userInfo" type="email" placeholder="Email" />
      <input type="submit" />
    </form>
  </article>
}

export default App
```

14. React 2 - Pokročilejší formulář - pracujeme s hodnotami z formuláře

Video: https://youtu.be/9HITQ_60FUU

Přidáme onChange, abychom mohli zachytit každou změnu, která se v políčku stane. Pomocí setFullName a setEmail uložíme tyto změny do proměnných fullName a email. Ve funkci formSubmit se zeptáme, jestli jsou vyplněné pomocí podmínky if. Pokud ano, tak hodnoty přidáme jako objekt do users.

```
import { useState } from "react"

const App = () => {
  const [fullName, setFullName] = useState("")
  const [email, setEmail] = useState("")
  const [users, setUsers] = useState([])

  const formSubmit = (event) => {
    event.preventDefault()

    if ( fullName && email ) {
      const oneUser = {fullName: fullName, email: email}
      setUsers( (users) => {
        return [...users, oneUser]
      })
    } else {
      console.log("Něco nebylo vyplněno!");
    }

    setFullName("")
    setEmail("")
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input
        className="userInfo"
        type="text"
        placeholder="Jméno"
        value={fullName}
        onChange={ (event) => setFullName(event.target.value) }
      />

      <input
        className="userInfo"
```

```
      type="email"
      placeholder="Email"
      value={email}
      onChange = { (event) => setEmail(event.target.value)}
    />

    <input type="submit" />
  </form>
</article>
}

export default App
```

15. React 2 - Pokročilejší formulář - vypisujeme informace do stránky

Video: <https://youtu.be/HLZJWueoUPM>

K výpisu do stránky použijeme funkci map.

```
return <article>
  <form onSubmit={formSubmit}>
    <input
      className="userInfo"
      type="text"
      placeholder="Jméno"
      value={fullName}
      onChange={ (event) => setFullName(event.target.value) }
    />

    <input
      className="userInfo"
      type="email"
      placeholder="Email"
      value={email}
      onChange = { (event) => setEmail(event.target.value) }
    />

    <input type="submit" />
  </form>

  {users.map( (oneUser, index) => {
    const {fullName, email} = oneUser

    return <div className="item" key={index}>
      <h4>{fullName}</h4>
      <p>{email}</p>
    </div>
  })}

</article>
```

Výsledek na stránce

David Šetek
david@gmail.com

Harry Potter
harry@gmail.com

16. React 2 - Pokročilejší formulář - přidání dalšího políčka

Video: <https://youtu.be/AmDYw722GCo>

Přidáme další políčko pro zadání věku. Podívejte se, kde všude se tato změna projeví.

```
import { useState } from "react"

const App = () => {
  const [fullName, setFullName] = useState("")
  const [email, setEmail] = useState("")
  const [age, setAge] = useState("")
  const [users, setUsers] = useState([])

  const formSubmit = (event) => {
    event.preventDefault()

    if ( fullName && email ) {
      const oneUser = {fullName: fullName, email: email, age: age}
      setUsers( (users) => {
        return [...users, oneUser]
      })
    } else {
      console.log("Něco nebylo vyplněno!");
    }

    setFullName("")
    setEmail("")
    setAge("")
  }

  return <article>
    <form onSubmit={formSubmit}>
      <input
        className="userInfo"
        type="text"
        placeholder="Jméno"
        value={fullName}
        onChange={ (event) => setFullName(event.target.value) }
      />

      <input
        className="userInfo"
        type="email"
```

```

        placeholder="Email"
        value={email}
        onChange = { (event) => setEmail(event.target.value)}
    />

    <input
        className="userInfo"
        type="text"
        placeholder="Věk"
        value={age}
        onChange={ (event) => setAge(event.target.value)}
    />

    <input type="submit" />
</form>

{users.map( (oneUser, index) => {
    const {fullName, email, age} = oneUser

    return <div className="item" key={index}>
        <h4>{fullName}</h4>
        <p>{email}</p>
        <p>{age}</p>
    </div>
    )}}

</article>
}

export default App

```

17. React 2 - Pokročilejší formulář - refactorujeme (zefektivňujeme) kód

Video: <https://youtu.be/Rvyx79qMDTk>

1. krok - místo 3 useState vytvoříme pouze jeden, který naplníme objektem, který se skládá z fullName, email a age.

```
const App = () => {  
  // const [fullName, setFullName] = useState("")  
  // const [email, setEmail] = useState("")  
  // const [age, setAge] = useState("")  
  
  const [oneUser, setOneUser] = useState({fullName: "", email: "", age:  
    ""})  
  const [users, setUsers] = useState([])
```

2. krok - při onChange budeme volat vždy stejnou funkci formChange. Každému políčku dáme speciální označení pomocí name.

```
<input  
  className="userInfo"  
  type="text"  
  placeholder="Jméno"  
  value={oneUser.fullName}  
  onChange={formChange}  
  name="fullName"  
/>  
  
<input  
  className="userInfo"  
  type="email"  
  placeholder="Email"  
  value={oneUser.email}  
  onChange={formChange}  
  name="email"  
/>  
  
<input  
  className="userInfo"  
  type="text"  
  placeholder="Věk"  
  value={oneUser.age}  
  onChange={formChange}  
  name="age"  
/>
```

3. krok - funkce formChange vždy zachytí, jaký input (jaké name) se mění a také vytáhne hodnotu. Tu zapíše do oneUser, u kterého zjistí podle name, jakou položku má updatovat.

```
const formChange = (event) => {  
  const name = event.target.name  
  const value = event.target.value  
  
  setOneUser({...oneUser, [name]: value })  
  
}
```

4. krok - vytvoříme proměnnou newUser a tu naplníme objektem se zadaným hodnotami z formuláře. Přidáme newUser do pole users.

```
const formSubmit = (event) => {  
  event.preventDefault()  
  
  if (oneUser.fullName && oneUser.email && oneUser.age) {  
    const newUser = { fullName: oneUser.fullName, email:  
oneUser.email, age: oneUser.age}  
    setUsers( (users) => {  
      return [...users, newUser]  
    })  
  } else {  
    console.log("Něco nebylo vyplněno!");  
  }  
  
  // setFullName("")  
  // setEmail("")  
  // setAge("")  
  
}
```

18. React 2 - Pokročilejší formulář - promazání formuláře po odeslání

Video: <https://youtu.be/LeUpelkGrus>

Promažeme formulář po odeslání.

```
const formSubmit = (event) => {  
  event.preventDefault()  
  
  if (oneUser.fullName && oneUser.email && oneUser.age) {  
    const newUser = { fullName: oneUser.fullName, email:  
oneUser.email, age: oneUser.age}  
    setUsers( (users) => {  
      return [...users, newUser]  
    })  
    setOneUser({ fullName: "", email: "", age: "" })  
  } else {  
    console.log("Něco nebylo vyplněno!");  
  }  
  
  // setFullName("")  
  // setEmail("")  
  // setAge("")  
  
}
```

19. React 2 - Pokročilejší formulář - jak jinak vytvořit unikátní identifikátor

Video: <https://youtu.be/v0E-55l6WjY>

App.js

Zelenou část kódu zakomentujeme a přidáme znovu proměnnou newUser. Do ní uložíme současného oneUser a k němu ještě přidáme id. ID odvodíme z času, které značí počet milisekund od 1. ledna 1970.

```
const formSubmit = (event) => {
  event.preventDefault()

  if (oneUser.fullName && oneUser.email && oneUser.age) {
    // const newUser = { fullName: oneUser.fullName, email:
oneUser.email, age: oneUser.age}
    const newUser = {...oneUser, id: new Date().getTime()}
    setUsers( (users) => {
      return [...users, newUser]
    })
    setOneUser({ fullName: "", email: "", age: "" })
  } else {
    console.log("Něco nebylo vyplněno!");
  }
}
```

App.js

V této části kódu jsme smazali proměnnou index. Doplníme ID jako unikátní identifikátor.

```
{users.map( (oneUser) => {
  const {id, fullName, email, age} = oneUser

  return <div className="item" key={id}>
    <h4>{fullName}</h4>
    <p>{email}</p>
    <p>{age}</p>
  </div>
})}
```

20. React 2 - Pokročilejší formulář - dynamické vlastnosti objektu

Video: <https://youtu.be/FctNySw8-d4>

```
// Dynamic object properties = dynamické vlastnosti objektu

const info = "cokolivJineho"

const person = {
  fullName: "David Šetek",
  email: "david@bradavice.cz"
}

// person.age = 40
person[info] = "red"

console.log(person)
```

21. React 2 - Lorem ipsum generátor - ukázka

Video: <https://youtu.be/QRluhcD7IKU>

Takto bude vypadat naše aplikace.

Lorem ipsum generator

Odstavce:

Vygenerovat

er adipiscing elit. Nulla turpis magna, cursus sit amet, suscipit a, interdum id, f

itation ullamco laboris nisi ut aliquip ex ea commodo consequat. Suspendisse n

o tempus molestie, porttitor ut, iaculis quis, sem. Morbi leo mi, nonummy eget

sagittis ultrices augue.

onsectetuer adipiscing elit. Maecenas fermentum, sem in pharetra pellentesque,

odio a lectus. In enim a arcu imperdiet malesuada.

n, orci vestibulum ullamcorper ultricies, lacus quam ultricies odio, vitae placera

e tincidunt ante vitae massa. Aliquam id dolor. Aliquam in lorem sit amet leo ac

at. Fusce consectetur risus a nunc. Morbi imperdiet, mauris ac auctor dictum,

is at pulvinar vulputate, erat libero tristique tellus, nec bibendum odio risus sit

tristique non, venenatis quis, ante.

22. React 2 - Lorem ipsum generátor - jak budeme postupovat

Video: <https://youtu.be/xFoUaA8vkMw>

Metoda slice:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/slice

23. React 2 - Lorem ipsum generátor - data a tvorba formuláře

Video: <https://youtu.be/LilazeJQqDg>

data.js

Tato data zkopírujeme do nově vytvořeného souboru data.js, který jsme umístili do složky src.

```
const text = [  
  "1Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla  
  turpis magna, cursus sit amet, suscipit a, interdum id, felis. Vivamus  
  ac leo pretium faucibus.",  
  "2Ut enim ad minim veniam, quis nostrud exercitation ullamco  
  laboris nisi ut aliquip ex ea commodo consequat. Suspendisse nisl. Sed  
  elit dui, pellentesque a, faucibus vel, interdum nec, diam. Sed vel  
  lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem.  
  Morbi leo mi, nonummy eget tristique non, rhoncus non leo. Suspendisse  
  sagittis ultrices augue.",  
  "3In convallis. Lorem ipsum dolor sit amet, consectetur adipiscing  
  elit. Maecenas fermentum, sem in pharetra pellentesque, velit turpis  
  volutpat ante, in pharetra metus odio a lectus. In enim a arcu  
  imperdiet malesuada.",  
  "4Morbi scelerisque luctus velit. Integer rutrum, orci vestibulum  
  ullamcorper ultricies, lacus quam ultricies odio, vitae placerat pede  
  sem sit amet enim. Vivamus porttitor turpis ac leo. Nunc tincidunt ante  
  vitae massa. Aliquam id dolor. Aliquam in lorem sit amet leo accumsan  
  lacinia.",  
  "5Integer vulputate sem a nibh rutrum consequat. Fusce consectetur  
  risus a nunc. Morbi imperdiet, mauris ac auctor dictum, nisl ligula  
  egetas nulla, et sollicitudin sem purus in lacus. In rutrum. Nullam  
  feugiat, turpis at pulvinar vulputate, erat libero tristique tellus,  
  nec bibendum odio risus sit amet ante. Etiam sapien elit, consequat  
  eget, tristique non, venenatis quis, ante."  
]  
  
export default text
```

App.js

Naimportujeme data z data.js a také useState, který v dalších kapitolách použijeme. Vytvoříme formulář a nastavíme, že až se odešle, tak se spustí funkce submitForm.

```
import data from "../data"
import { useState } from "react"

const App = () => {

  const submitForm = (e) => {
    e.preventDefault()
    console.log("Odeslán formulář")
  }

  return (
    <section>
      <h1>Lorem ipsum generátor</h1>
      <form onSubmit={submitForm}>
        <label htmlFor="paragraphs">Odstavce:</label>
        <input type="number" id="paragraphs"/><br />
        <input type="submit" value="Vygenerovat"/>
      </form>
    </section>
  )
}

export default App
```

Výsledek na stránce

Lorem ipsum generátor

Odstavce:

24. React 2 - Lorem ipsum generátor - zachytáváme hodnotu z políčka

Video: <https://youtu.be/2a7TBZLkDgo>

Vytvoříme proměnnou count přes useState. Do této proměnné budeme ukládat hodnotu z políčka ve formuláři. Takže nastavíme u políčka onChange, které se spouští při každé změně. Vytáhneme obsah políčka pomocí e.target.value a pomocí setCount nastavíme do proměnné count. Po odeslání formuláře převedeme obsah count na celé číslo neboli integer (count obsahuje string) a vypíšeme do konzole typ a přesvědčíme se, jestli je to skutečně číslo.

```
import data from "../data"
import { useState } from "react"

const App = () => {
  const [count, setCount] = useState(0)

  const submitForm = (e) => {
    e.preventDefault()
    const amount = parseInt(count)
    console.log(typeof(amount))
  }

  return (
    <section>
      <h1>Lorem ipsum generátor</h1>
      <form onSubmit={submitForm}>
        <label htmlFor="paragraphs">Odstavce:</label>
        <input type="number" id="paragraphs" onChange={ (e) =>
{setCount(e.target.value)} }/><br />
        <input type="submit" value="Vygenerovat"/>
      </form>
    </section>
  )
}

export default App
```

25. React 2 - Lorem ipsum generátor - vypisujeme odstavce

Video: <https://youtu.be/iZZ70Nvd4sk>

Vytvoříme proměnnou paragraphs, do které budeme ukládat potřebný počet odstavců podle toho, co uživatel zadá do políčka. Když do políčka zadá číslo 3, tak v paragraphs budou 3 odstavce atd.

Do paragraphs pomocí setParagraphs uložíme odstavce. Dole v return je pomocí funkce map vypíšeme.

```
const App = () => {
  const [count, setCount] = useState(0)
  const [paragraphs, setParagraphs] = useState([])

  const submitForm = (e) => {
    e.preventDefault()
    const amount = parseInt(count)

    setParagraphs(data.slice(0, amount))
  }

  return (
    <section>
      <h1>Lorem ipsum generátor</h1>
      <form onSubmit={submitForm}>
        <label htmlFor="paragraphs">Odstavce:</label>
        <input type="number" id="paragraphs" onChange={ (e) =>
{setCount(e.target.value)} }/><br />
        <input type="submit" value="Vygenerovat"/>
      </form>

      <article>
        {
          paragraphs.map( (oneParagraph, index) => {
            return <p key={index}>{oneParagraph}</p>
          })
        }
      </article>
    </section>
  )
}

export default App
```

Výsledek na stránce - pokud do políčka zadáme číslo a dáme odeslat, tak se vygeneruje odpovídající počet odstavců.

Lorem ipsum generátor

Odstavce:

1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla turpis magna, cursus sit amet, suscipit a, interdum id, felis. Vivamus ac leo pretium faucibus.

2 Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Suspendisse nisl. Sed elit dui, pellentesque a, faucibus vel, interdum nec, diam. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Morbi leo mi, nonummy eget tristique non, rhoncus non leo. Suspendisse sagittis ultrices augue.

26. React 2 - Lorem ipsum generátor - řešíme záporné a moc velké hodnoty

Video: https://youtu.be/aEYgbtq_n0c

Přidáme kontrolu na krajní hodnoty. Pokud někdo zadá např. -5 odstavců, tak pokud bude hodnota menší než nula, tak amount nastavíme na 1. Naopak, pokud někdo zadá více odstavců, než máme k dispozici, tak se amount nastaví na 5.

Pozor - jakmile do amount přiřadíme novou hodnotu např. 1, tak musíme nahoře amount přepsat z const na let, aby přiřazení bylo možné.

```
import data from "../data"
import { useState } from "react"

const App = () => {
  const [count, setCount] = useState(0)
  const [paragraphs, setParagraphs] = useState([])

  const submitForm = (e) => {
    e.preventDefault()
    let amount = parseInt(count)

    // kontrola krajních hodnot
    if (amount < 0){
      amount = 1
    } else if (amount > 5){
      amount = 5
    }

    setParagraphs(data.slice(0, amount))
  }

  return (
    <section>
      <h1>Lorem ipsum generátor</h1>
      <form onSubmit={submitForm}>
        <label htmlFor="paragraphs">Odstavce:</label>
        <input type="number" id="paragraphs" onChange={ (e) =>
{setCount(e.target.value)} }/><br />
        <input type="submit" value="Vygenerovat"/>
      </form>

      <article>
        {
          paragraphs.map( (oneParagraph, index) => {
```

```
        return <p key={index}>{oneParagraph}</p>
      })
    }
  </article>
</section>
)
}

export default App
```

27. React 2 - Lorem ipsum generátor - alternativní přístup přes metodu filter

Video: <https://youtu.be/H3L4mpKUSXU>

Jedna možnost je použít slice pro získání odpovídajícího počtu odstavců nebo můžeme použít metodu filter. Filter je tedy alternativa ke slice. Je na vás, kterou možnost použijete.

```
const submitForm = (e) => {
  e.preventDefault()
  let amount = parseInt(count)

  // kontrola krajních hodnot
  if (amount < 0){
    amount = 1
  } else if (amount > 5){
    amount = 5
  }

  // const resultParagraphs = data.filter( (oneParagraph, index) => {
  //   return index < amount
  // })
  // setParagraphs(resultParagraphs)

  setParagraphs(data.slice(0, amount))
}
```

28. React 2 - Lorem ipsum generátor - stylování

Video: <https://youtu.be/4DSVkxlnK4w>

App.js

Celou sekci označíme pomocí classy form-section.

```
return (  
  <section className="form-section">  
    <h1>Lorem ipsum generátor</h1>  
    <form onSubmit={submitForm}>
```

Index.css

```
body {  
  margin: 0;  
  padding: 0;  
  background-color: #141414;  
  color: #e5e5e5;  
}  
  
.form-section {  
  height: 100vh;  
  display: flex;  
  flex-direction: column;  
  justify-content: flex-start;  
  align-items: center;  
  text-align: center;  
}  
  
.form-section h1 {  
  margin-top: 0px;  
  padding-top: 50px;  
}  
  
.form-section label {  
  margin-right: 10px;  
}  
  
.form-section input[type="number"] {  
  margin-bottom: 20px;  
  font-size: 18px;  
  text-align: center;  
  outline: none;  
}
```

```
.form-section input[type="submit"] {  
  background-color: #e50914;  
  color: #e5e5e5;  
  border: none;  
  padding: 5px 10px;  
  font-size: 18px;  
}
```

Výsledek na stránce.

Lorem ipsum generátor

Odstavce:

Vygenerovat

1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla turpis magna, cursus sit amet, suscipit a, interdum id, felis. Vivamus ac leo pretium faucibus.

2 Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Suspendisse nisl. Sed elit dui, pellentesque a, faucibus vel, interdum nec, diam. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Morbi leo mi, nonummy eget tristique non, rhoncus non leo. Suspendisse sagittis ultrices augue.

3 In convallis. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas fermentum, sem in pharetra pellentesque, velit turpis volutpat ante, in pharetra metus odio a lectus. In enim a arcu imperdiet malesuada.

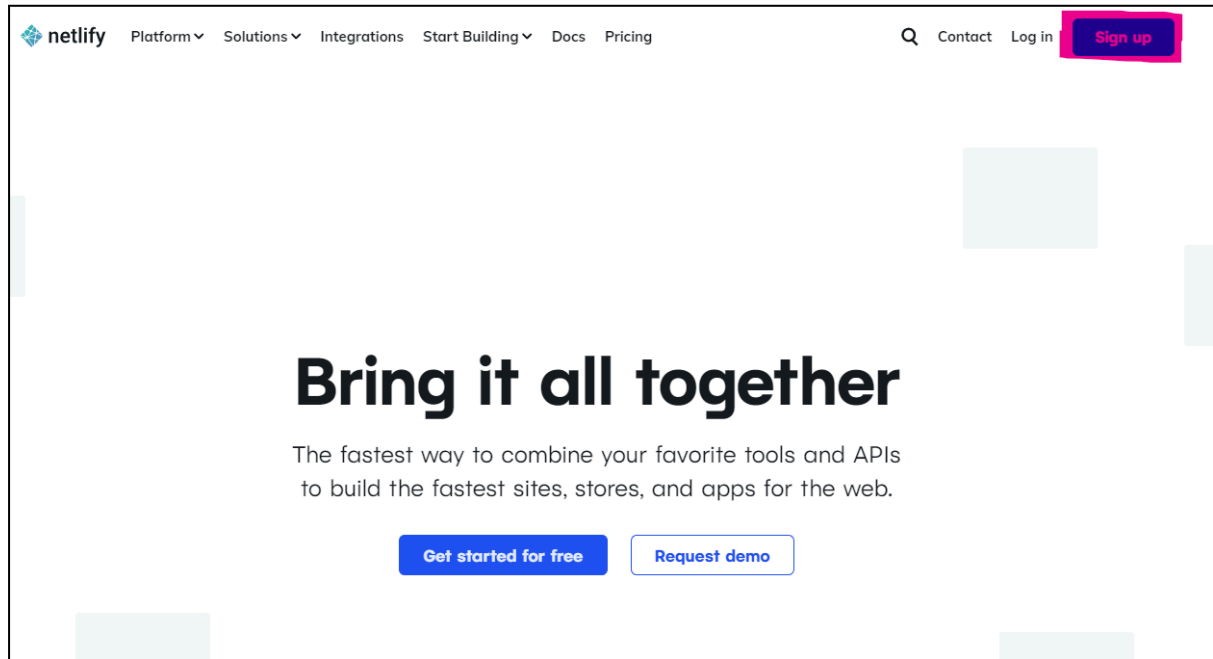
29. React 2 - Publikujeme projekt v reactu na internet

Video: <https://youtu.be/Dvm9LzziuAU>

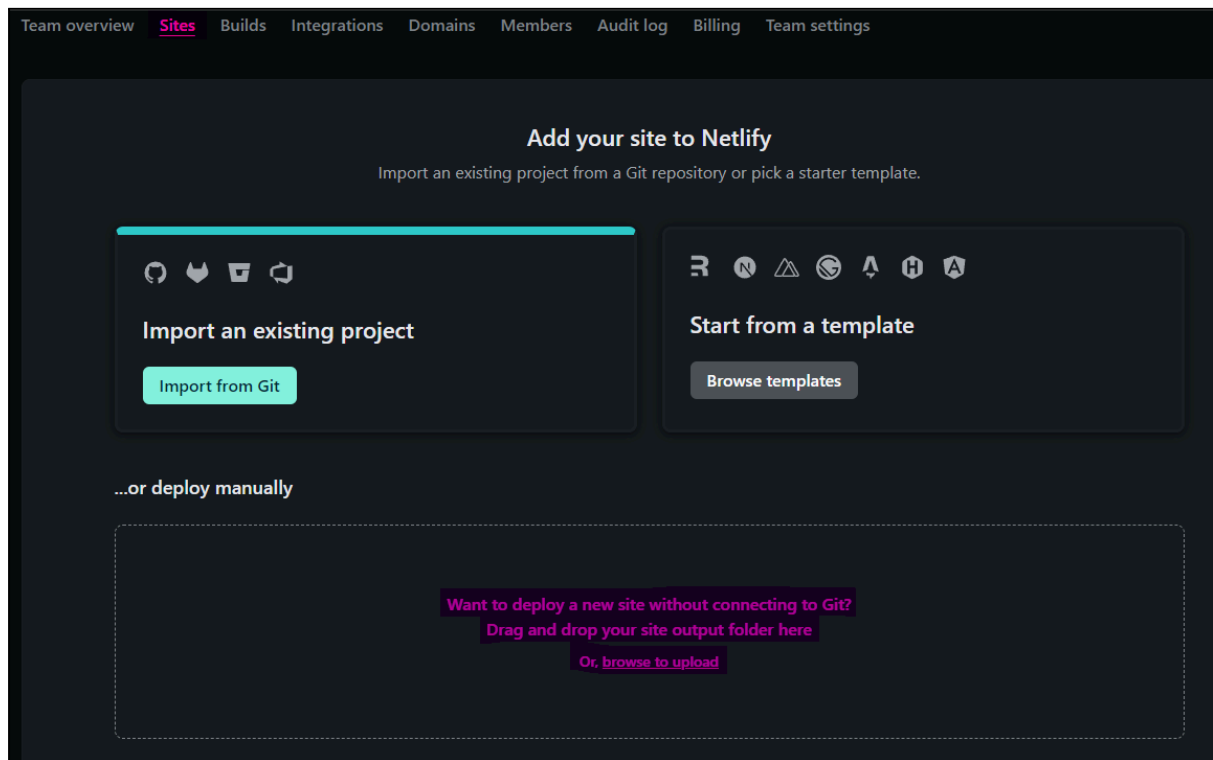
Stránka, kam projekt nahráváme:

<https://www.netlify.com/>

Zde se zaregistrujeme.

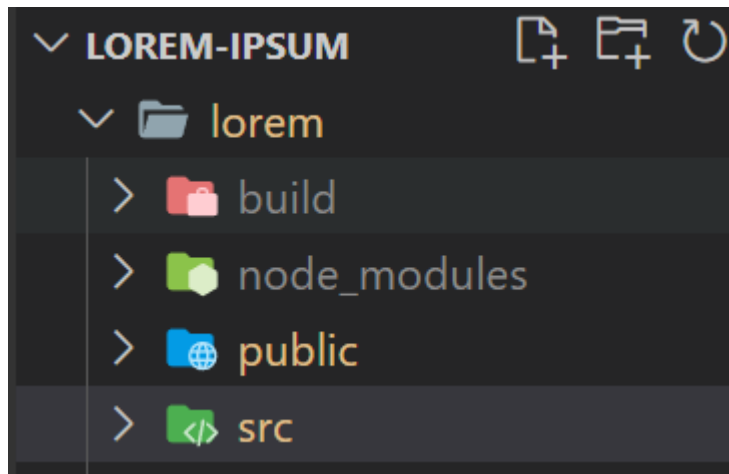


Sem přetáhneme složku build



Jak získáme složku build. V editoru kódu (např. VS code) dáme příkaz:
npm run build

Vytvoří se složka build a tu přetáhneme na Netlify.



30. React 2 - useRef

Video: <https://youtu.be/H7O1j5g3DKU>

Ve video jsme si zakládali nový projekt, takže opět nastavíme běžným způsobem index.html a index.js.

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result"></div>
</body>
</html>
```

Index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

App.js

Nyní si vysvětlíme useRef. Pokud ho chceme použít, tak ho musíme nainportovat. Hook useRef nejdříve vytvoříme (např. refInput) a potom ho umístíme k nějakému HTML tagu. Takže do refInput se nám uloží objekt, kde díky current můžeme přistoupit přímo k HTML tagu a dělat si s ním cokoli, co uznáme za vhodné. Např. mu změnit styly, textový obsah atd.

```
import { useRef } from "react"

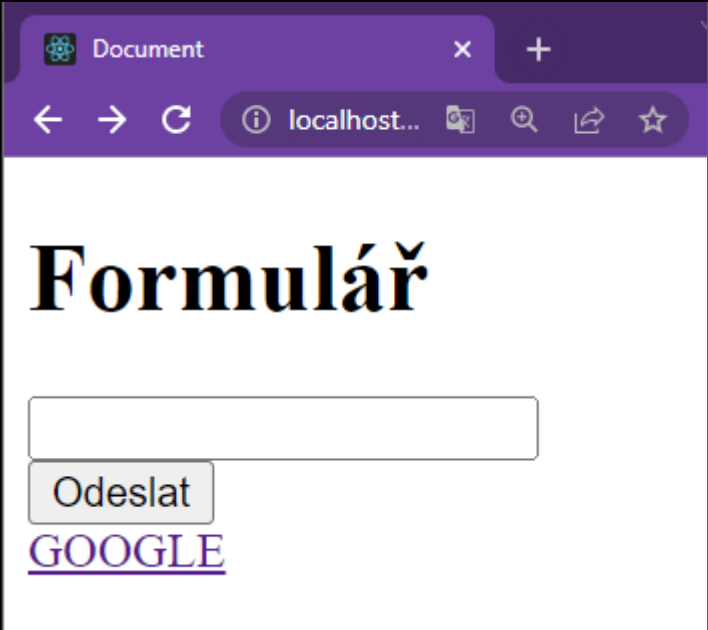
const App = () => {
  const refInput = useRef(null)
  const refHeading = useRef(null)
  const refLink = useRef(null)

  const formSubmit = (e) => {
    e.preventDefault()
    // console.log(refInput.current.value)
    // console.log(refInput.current.style.backgroundColor = "red")
    // console.log(refInput.current.style.color = "white")
    // console.log(refHeading.current.style.color = "orange")
    // console.log(refHeading.current.textContent = "nový text")
    // console.log(refLink.current.textContent = "SEZNAM.CZ")
    // console.log(refLink.current.href = "https://www.seznam.cz")
  }

  return <div>
    <h1 ref={refHeading}>Formulář</h1>
    <form onSubmit={formSubmit}>
      <input type="text" ref={refInput} /><br />
      <input type="submit"/>
    </form>
    <a href="https://www.google.com" ref={refLink}>GOOGLE</a>
  </div>
}

export default App
```

Výsledek na stránce



The screenshot shows a web browser window with a purple header bar. The tab is labeled 'Document'. The address bar shows 'localhost...'. The main content area has a large heading 'Formulář' in a black serif font. Below the heading is a white rectangular input field. Underneath the input field is a grey button with the text 'Odeslat'. At the bottom of the form area is a purple underlined link that says 'GOOGLE'.

Document

localhost...

Formulář

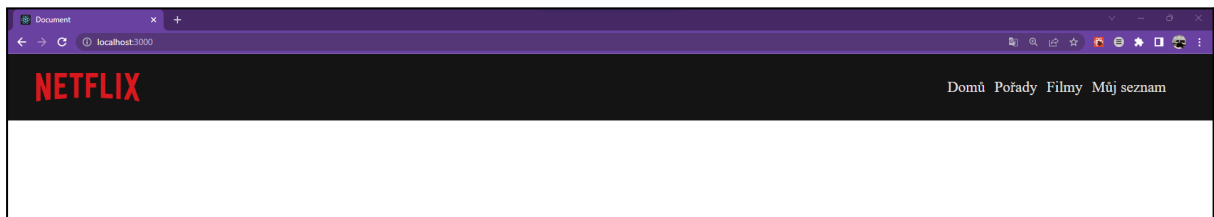
Odeslat

[GOOGLE](#)

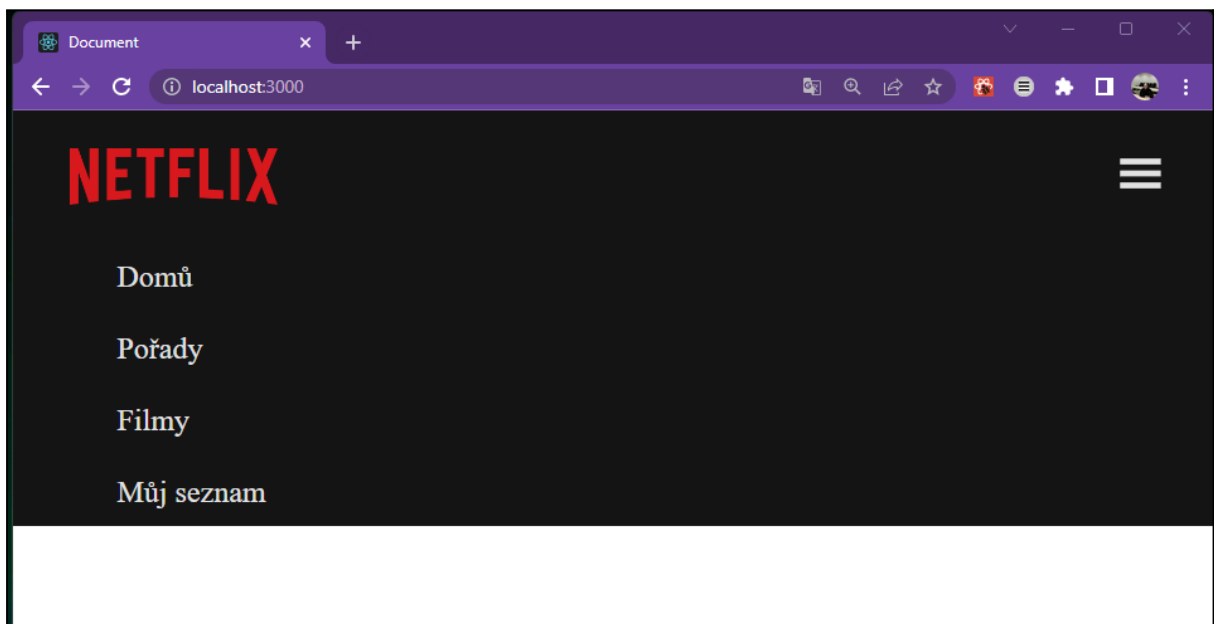
31. React 2 - Projekt menu - ukázka

Video: <https://youtu.be/r7sM9GalZz0>

Takto bude vypadat menu na větší obrazovce.



Takto na menší obrazovce



32. React 2 - Projekt menu - základní nastavení

Video: https://youtu.be/8eWZL_h-ZSQ

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result"></div>
</body>
</html>
```

App.js

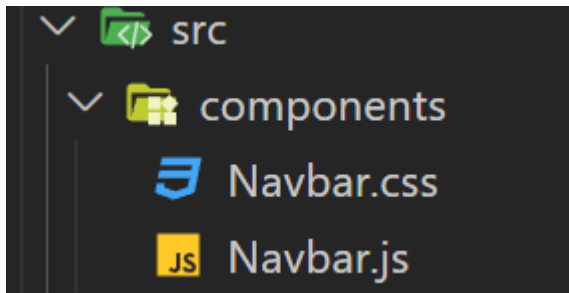
```
const App = () => {
  return (
    <Navbar />
  )
}

export default App
```

Index.css

```
body {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

Vytvoříme ve složce src složku components a tam založíme Navbar.js a Navbar.css



App.js

Do App.js Navbar.js naimportujeme a rovnou vypíšeme do stránky.

```
import Navbar from "../components/Navbar"

const App = () => {
  return (
    <Navbar />
  )
}

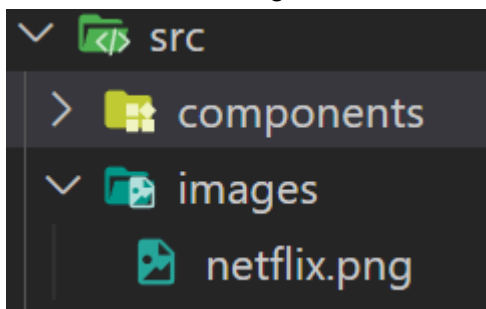
export default App
```

Odkazy

Logo Netflixu

https://drive.google.com/drive/folders/1AvuFn3Mpb3hbkewbtOQgaWeryE_rxYI?usp=sharing

Založíme složku images a do ní vložíme logo Netflixu



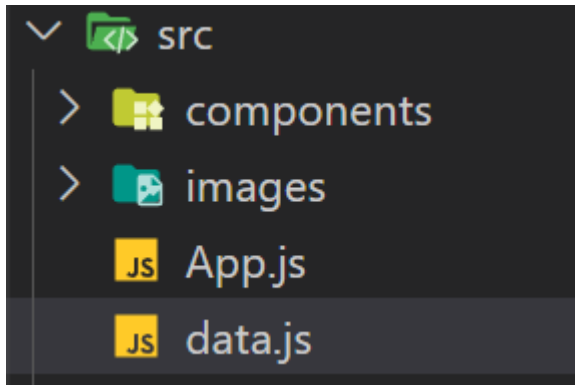
Instalace ikonek

<https://react-icons.github.io/react-icons/>

Game Icons

<https://react-icons.github.io/react-icons/icons?name=gi>

Založíme soubor data.js ve složce src.



data.js

Do souboru vložíme tato data včetně exportu na konci

```
const links = [
  {
    id: 1,
    url: "/",
    text: "Domů",
  },
  {
    id: 2,
    url: "/porady",
    text: "Pořady",
  },
  {
    id: 3,
    url: "/filmy",
    text: "Filmy",
  },
  {
    id: 4,
    url: "/mujseznam",
    text: "Můj seznam",
  },
]

export default links
```

Výsledná podoba Navbar.js

```
import "./Navbar.css"
import logo from "../images/netflix.png"
```

```
import { GiHamburgerMenu } from "react-icons/gi";
import links from "../data"

const Navbar = () => {
  return (
    <div>Navbar</div>
  )
}

export default Navbar
```

33. React 2 - Projekt menu - Obsah navigace

Video: https://youtu.be/_rdVLjiT9_0

Vytvoříme strukturu navigace a použijeme nainportované logo a ikonku. Poté také data k vypsání položek menu.

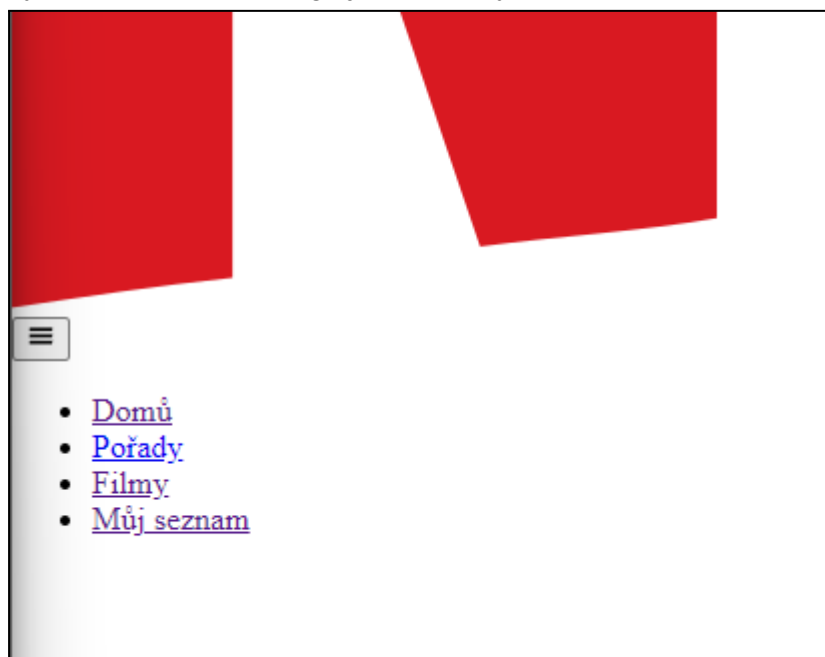
```
import './Navbar.css'
import logo from '../images/netflix.png'
import { GiHamburgerMenu } from 'react-icons/gi';
import links from '../data'

const Navbar = () => {
  return <nav>
    <div className="navigation">
      <div className="nav-header">
        <img src={logo} alt="" />
        <button>
          <GiHamburgerMenu />
        </button>
      </div>
      <div>
        <ul>
          {
            links.map( (oneLink) => {
              const {id, text, url} = oneLink

              return <li key={id}>
                <a href={url}>{text}</a>
              </li>
            })
          }
        </ul>
      </div>
    </div>
  </nav>
}

export default Navbar
```

Výsledek na stránce (logo je veliké - vyřešíme pomocí CSS dále)



34. React 2 - Projekt menu - stylování 1. část

Video: <https://youtu.be/9URmotQcUfc>

Navbar.js

Přidáme k divu, který obsahuje seznam položek menu, className.

```
<div className="nav-list">
  <ul>
    {
      links.map( (oneLink) => {
        const {id, text, url} = oneLink

        return <li key={id}>
          <a href={url}>{text}</a>
        </li>
      })
    }
  </ul>
</div>
```

Navbar.css

```
/* Do 700px */
nav {
  background-color: #141414;
}

.navigation {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
}

.nav-header {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  min-height: 70px;
}

.nav-header img {
  width: 110px;
  margin-left: 30px;
}
```

```

}

.nav-header button {
  display: none;
}

.nav-list {
  display: flex;
  flex-direction: row;
  align-items: center;
}

.nav-list ul {
  margin-right: 50px;
}

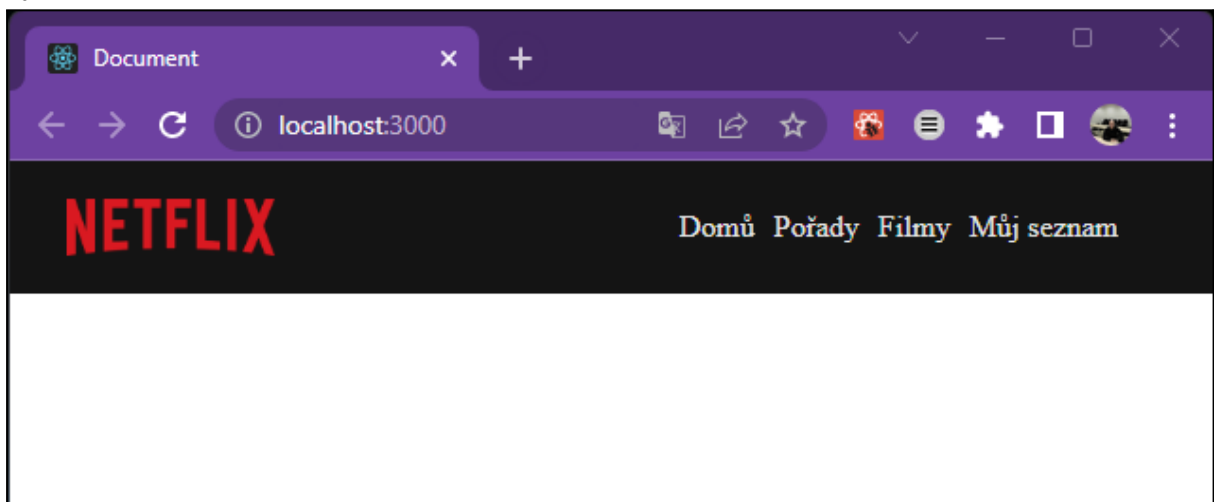
.nav-list li {
  display: inline-block;
  margin-left: 10px;
}

.nav-list a {
  text-decoration: none;
  color: #e5e5e5;
}

/* Od 700px a méně */

```

Výsledek na stránce



35. React 2 - Projekt menu - stylování 2. část

Video: <https://youtu.be/JBKY04nLhjQ>

Navbar.js

Přidáme classu přímo k ikonce, abychom ji mohli v CSS nastýlovat.

```
<button>
  <GiHamburgerMenu className="hamburger-icon"/>
</button>
```

Navbar.css

```
/* 700px a méně */
@media (max-width: 700px){
  .navigation {
    flex-direction: column;
    justify-content: center;
    align-items: flex-start;
  }

  .nav-header {
    width: 100%;
  }

  .nav-header button {
    display: block;
    background-color: transparent;
    border: none;
  }

  .hamburger-icon {
    color: #e5e5e5;
    font-size: 25px;
    margin-right: 20px;
  }

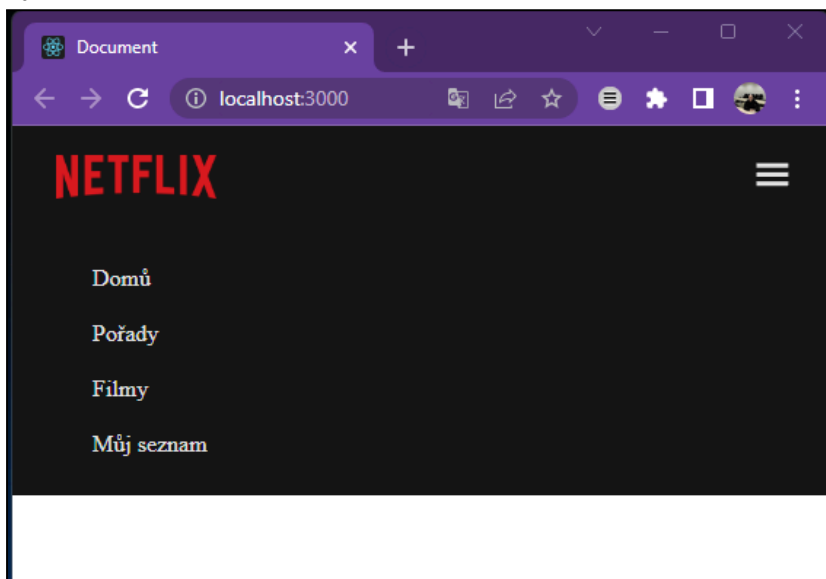
  .nav-list li {
    width: 100%;
    padding: 10px 5px;
  }

  /* Classy pro použití v reactu */
  .hide {
    height: 0px;
    overflow: hidden;
  }
}
```

```
        transition: all 0.1s linear;
    }

    .show {
        height: 150px;
        transition: all 0.1s linear;
        /* background-color: #141414;
        width: 100%; */
    }
}
```

Výsledek na stránce



36. React 2 - Projekt menu - vysouvání a zasouvání menu

Video: <https://youtu.be/UtKAiyAT6Lg>

Navbar.js

Naimportujeme useState a vytvoříme proměnnou showMenu. Do showMenu nastavíme jako výchozí hodnotu false.

Pokud klikneme na tlačítko (ikonku), tak změníme hodnotu v showMenu na opačnou. Pokud je v ní false, tak ji změníme na true. Pokud je v ní true, tak to změníme na false.

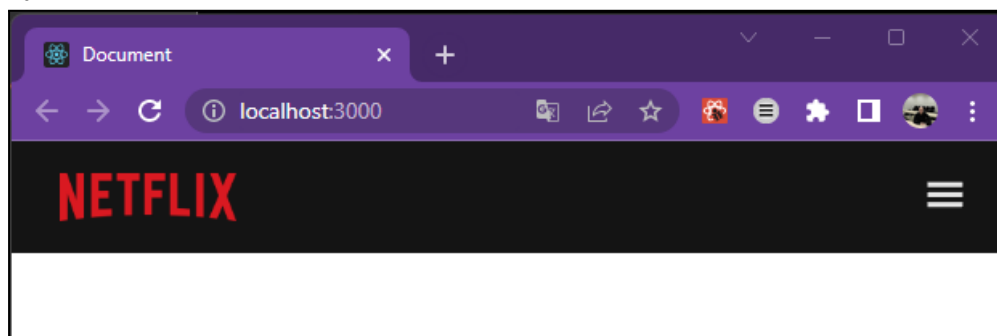
Pokud je v showMenu true, tak se k divu, který obaluje celý seznam, přidají classy nav-list a show. Pokud je v showMenu false, tak se přidají nav-list a hide. Classy show a hide způsobují objevování a mizení položek menu.

```
import "../Navbar.css"
import logo from "../images/netflix.png"
import { GiHamburgerMenu } from "react-icons/gi";
import links from "../data"
import { useState } from "react"

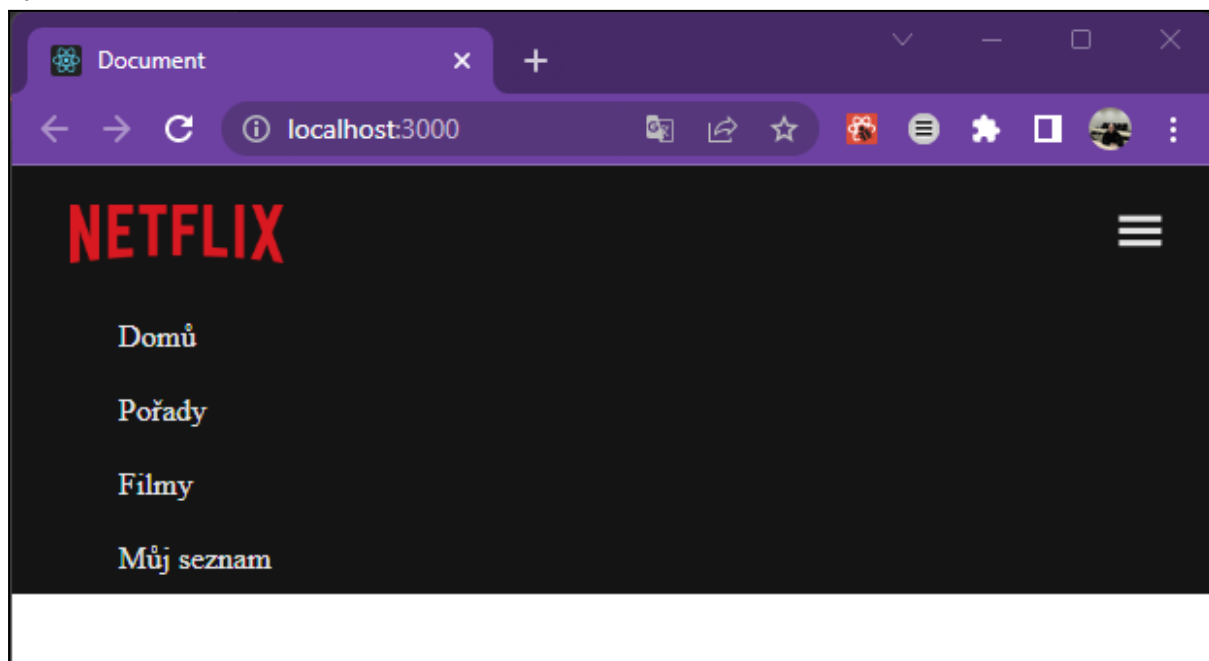
const Navbar = () => {
  const [showMenu, setShowMenu] = useState(false)

  return <nav>
    <div className="navigation">
      <div className="nav-header">
        <img src={logo} alt="" />
        <button onClick={() => setShowMenu(!showMenu)} >
          <GiHamburgerMenu className="hamburger-icon"/>
        </button>
      </div>
      <div className={` ${showMenu ? "nav-list show" : "nav-list hide"} `>
        <ul>
```

Výsledek na stránce před kliknutím



Výsledek na stránce po kliknutí



37. React 2 - Projekt menu - opravujeme chybu

Video: <https://youtu.be/Xg4hwiUhQXI>

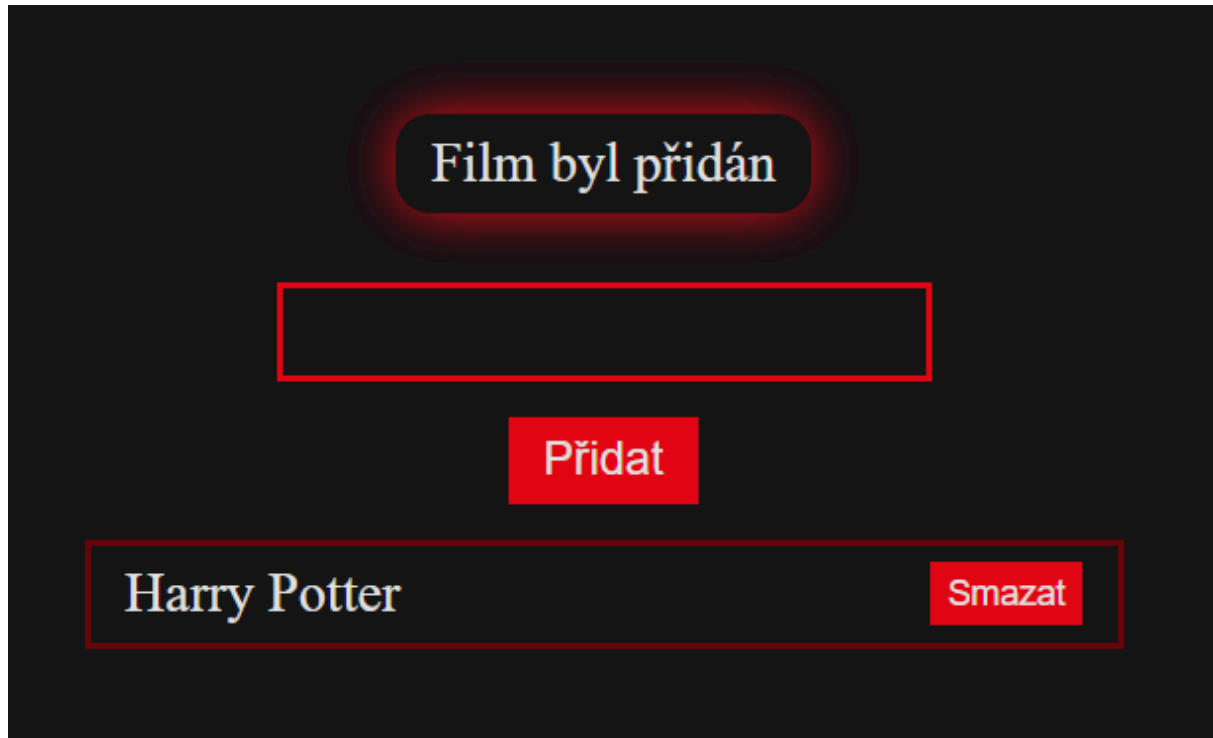
Aby při vyjždění nevyjžděli položky menu shora přes logo, tak odstraníme transition u show.

```
.show {  
  height: 150px;  
  /* transition: all 0.1s linear; */  
  /* background-color: #141414; */  
  width: 100%; */  
}
```

38. React 2 - useReducer a notifikace - ukázka

Video: <https://youtu.be/gfVxDRg-KSg>

Takto bude vypadat naše aplikace.



39. React 2 - useReducer a notifikace - základní nastavení a formulář

Video: <https://youtu.be/BOy2DN-f-mU>

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result"></div>
</body>
</html>
```

Index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

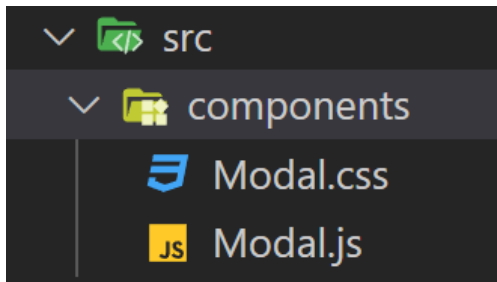
data.js

Nezapomeňte zkopírovat i poslední řádek s exportem.

```
const movies = [
  { id: 1, name: 'Čung_e' },
  { id: 2, name: 'Zátah 2' },
  { id: 3, name: 'Zrození šampióna' },
  { id: 4, name: 'D-tox' },
];

export default movies
```

Ve složce src vytvoříme složku components a v ní soubory Modal.js a Modal.css



Modal.js

```
import "../Modal.css"

const Modal = () => {
  return (
    <div>Film byl přidán</div>
  )
}

export default Modal
```

App.js

```
import Modal from "../components/Modal"
import data from "../data"
import { useState } from "react"

const App = () => {

  const submitForm = (e) => {
    e.preventDefault()
  }

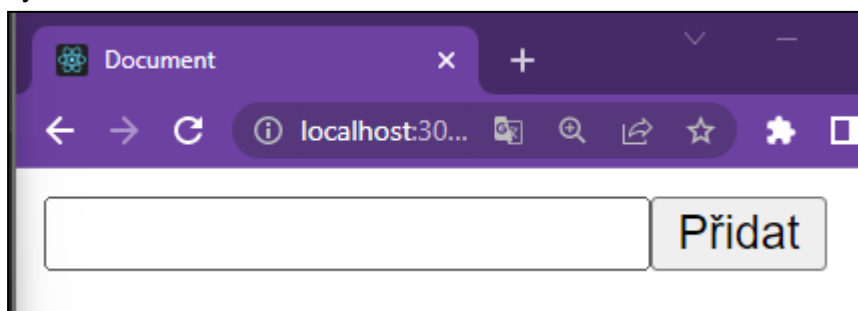
  return <section>
    <form onSubmit={submitForm}>
      <input type="text" />
      <input type="submit" value="Přidat" />
    </form>
  </section>

}

export default App
```

Soubory index.css a Modal.css jsou prázdné

Výsledek na stránce



40. React 2 - useReducer a notifikace - useState a vypsání filmů

Video: <https://youtu.be/CRi9YLQsbKQ>

App.js

Pomocí useState nastavíme 3 proměnné - movies (data o všech filmech), showNotification (řídí, jestli má být notifikace viditelná nebo ne) a movieName (název filmu zadáný v políčku).

Upravíme políčko, aby do movieName ukládalo to, co uživatel píše do políčka.

Necháme dole pod formulářem vypsát obsah proměnné movies.

```
import Modal from "../components/Modal"
import data from "../data"
import { useState } from "react"

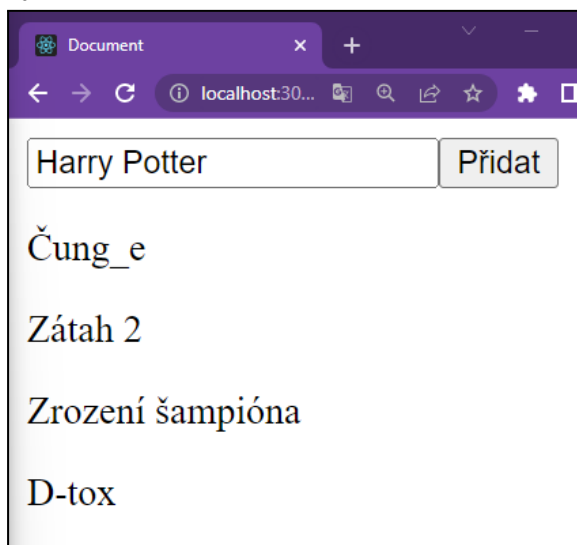
const App = () => {
  const [movies, setMovies] = useState(data)
  const [showNotification, setShowNotification] = useState(false)
  const [movieName, setMovieName] = useState("")

  const submitForm = (e) => {
    e.preventDefault()
  }

  return <section>
    <form onSubmit={submitForm}>
      <input type="text" onChange={ (e) => setMovieName(e.target.value) }/>
      <input type="submit" value="Přidat" />
    </form>
    <div>
      {movies.map( (oneMovie) => {
        return <div key={oneMovie.id}>
          <p>{oneMovie.name}</p>
        </div>
      } )}
    </div>
  </section>
}

export default App
```

Výsledek na stránce



The screenshot shows a web browser window with a purple header. The address bar displays 'localhost:30...'. The main content area contains a form with a text input field containing 'Harry Potter' and a button labeled 'Přidat'. Below the input field is a list of items: 'Čung_e', 'Zátah 2', 'Zrození šampióna', and 'D-tox'.

Document

localhost:30...

Harry Potter Přidat

Čung_e

Zátah 2

Zrození šampióna

D-tox

41. React 2 - useReducer a notifikace - vyskakování notifikace

Video: <https://youtu.be/m791JQm5y4Q>

App.js

Před formulář doplníme podmínku, že pokud je showNotification false, tak modální okno nezobrazuj. Pokud je true, tak zobraz.

Po odeslání formuláře (ve funkci submitForm) zadáme, že pokud movieName existuje, tak ať se showNotification nastaví na true. Jinak, ať se nastaví na false.

```
import Modal from "../components/Modal"
import data from "../data"
import { useState } from "react"

const App = () => {
  const [movies, setMovies] = useState(data)
  const [showNotification, setShowNotification] = useState(false)
  const [movieName, setMovieName] = useState("")

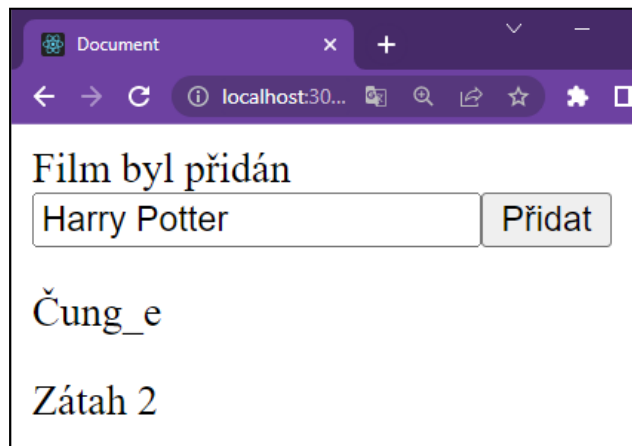
  const submitForm = (e) => {
    e.preventDefault()

    if (movieName) {
      setShowNotification(true)
    } else {
      setShowNotification(false)
    }
  }

  return <section>
    {showNotification && <Modal />}
    <form onSubmit={submitForm}>
      <input type="text" onChange={ (e) => setMovieName(e.target.value) }/>
      <input type="submit" value="Přidat" />
    </form>
  </div>
  {movies.map( (oneMovie) => {
    return <div key={oneMovie.id}>
      <p>{oneMovie.name}</p>
    </div>
  } )}
  </div>
</section>
}
```

```
export default App
```

Výsledek na stránce



42. React 2 - useReducer a notifikace - jdeme na useReducer

Video: https://youtu.be/YN6OUTqC_w8

App.js

Naimportujeme useReducer. Vytvoříme proměnnou defaultState, kterou pomocí useReducer uložíme do proměnné state. Ke všem třem vlastnostem (movies, showNotification, notificationContent) přistupujeme přes state. Například state.movies atd. Proto musíme na dvou místech přepsat state.showNotification a state.movies.

```
import Modal from "../components/Modal"
import data from "../data"
import { useState, useReducer } from "react"

const reducer = () => {

}

const defaultState = {
  movies: [],
  showNotification: false,
  notificationContent: ""
}

const App = () => {
  // const [movies, setMovies] = useState(data)
  // const [showNotification, setShowNotification] = useState(false)
  const [movieName, setMovieName] = useState("")
  const [state, dispatch] = useReducer(reducer, defaultState)

  const submitForm = (e) => {
    e.preventDefault()

    if (movieName) {
      // VYMAZÁNO
    } else {
      // VYMAZÁNO
    }
  }

  return <section>
    {state.showNotification && <Modal />}
    <form onSubmit={submitForm}>
      <input type="text" onChange={ (e) => setMovieName(e.target.value)} />
    </form>
  </section>
}
```

```
      <input type="submit" value="Přidat" />
    </form>
    <div>
      {state.movies.map( (oneMovie) => {
        return <div key={oneMovie.id}>
          <p>{oneMovie.name}</p>
        </div>
      } )}
    </div>
  </section>
}

export default App
```

43. React 2 - useReducer a notifikace - posíláme text do druhé komponenty

Video: <https://youtu.be/tyDwgJFHEZg>

App.js

Nyní vezmeme text z defaultState a pošleme ho do componenty Modal.

```
return <section>
  {state.showNotification && <Modal
notifContent={state.notificationContent} />}
  <form onSubmit={submitForm}>
    <input type="text" onChange={ (e) => setMovieName(e.target.value) }/>
    <input type="submit" value="Přidat" />
  </form>
```

Modal.js

Zde si text převezmeme pomocí názvu notifContent a zobrazíme v komponentě v divech.

```
import "../Modal.css"

const Modal = ( { notifContent } ) => {
  return (
    <div>{notifContent}</div>
  )
}

export default Modal
```

44. React 2 - useReducer a notifikace - dispatch a action.type

Video: <https://youtu.be/Y7VBRipt-Xo>

Do reducer se vždy posílají dvě věci state a action. State v sobě drží současný stav a action název akce, kterou zadáváme u dispatch. V našem případě název "ADD_MOVIE". V reduceru stanovíme, co se má při akci "ADD_MOVIE" stát.

Reducer musí vždy vrátet stav. Nejčastěji současný stav ve state upravíte a vrátíte ho.

```
const reducer = (state, action) => {
  console.log(state, action)

  if (action.type === "ADD_MOVIE"){
    return {
      ...state,
      movies: data,
      showNotification: true,
      notificationContent: "Film byl přidán"
    }
  }

  return state
}

const defaultState = {
  movies: [],
  showNotification: false,
  notificationContent: ""
}

const App = () => {
  // const [movies, setMovies] = useState(data)
  // const [showNotification, setShowNotification] = useState(false)
  const [movieName, setMovieName] = useState("")
  const [state, dispatch] = useReducer(reducer, defaultState)

  const submitForm = (e) => {
    e.preventDefault()

    if (movieName) {
      dispatch({type: "ADD_MOVIE"})
    } else {

    }
  }
}
```

45. React 2 - useReducer a notifikace - posíláme do reducer nový film

Video: <https://youtu.be/A7vzgijQ-Is>

App.js

Ve funkci po odeslání formuláře si připravíme proměnnou newMovie, kterou naplním objektem, který obsahuje nový film včetně jeho ID.

Přes payload pošleme newMovie do funkce reducer.

```
const submitForm = (e) => {
  e.preventDefault()

  if (movieName) {
    const newMovie = {id: new Date().getTime(), name: movieName}
    dispatch( {type: "ADD_MOVIE", payload: newMovie} )
  } else {

  }
}
```

App.js

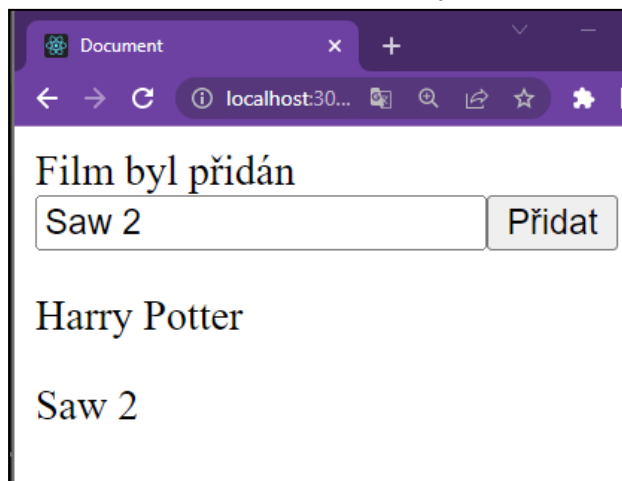
V reducer přebereme newMovie jako action.payload. Vytvoříme aktuální seznam všech filmů v proměnné newMovies a toto pole nastavíme jako hodnotu pro movies - tedy ve stavu, který jako výsledek reducer vrací.

```
const reducer = (state, action) => {

  if (action.type === "ADD_MOVIE"){
    const newMovies = [...state.movies, action.payload]
    return {
      ...state,
      movies: newMovies,
      showNotification: true,
      notificationContent: "Film byl přidán"
    }
  }

  return state
}
```

Pokud přidáme film, tak se nám vypíše dolů pod formulář.



The screenshot shows a web browser window with a purple header. The address bar displays 'localhost:30...'. The main content area contains a form with a text input field containing 'Saw 2' and a button labeled 'Přidat'. Below the form, the text 'Film byl přidán' is displayed. Underneath this, there is a list of movies: 'Harry Potter' and 'Saw 2'.

Film byl přidán

Saw 2 Přidat

Harry Potter

Saw 2

46. React 2 - useReducer a notifikace - nezadán žádná název filmu

Video: <https://youtu.be/z-tNBOqm6gw>

App.js

Pokud není zadáné žádné jméno filmu (políčko nebylo vyplněno), tak spust' dispatch s typem "NO_MOVIE_NAME" a pošli to do reducer.

```
const submitForm = (e) => {
  e.preventDefault()

  if (movieName) {
    const newMovie = {id: new Date().getTime(), name: movieName}
    dispatch( {type: "ADD_MOVIE", payload: newMovie} )
  } else {
    dispatch( {type: "NO_MOVIE_NAME"} )
  }
}
```

App.js

Nastavíme podmínku, že pokud je type "NO_MOVIE_NAME", tak přepni showNotification na true a notificationContent nastav na "Zadejte film".

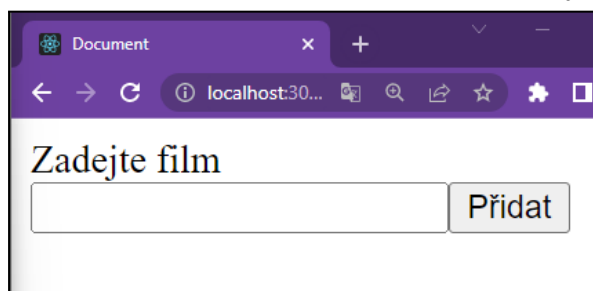
```
const reducer = (state, action) => {

  if (action.type === "ADD_MOVIE") {
    const newMovies = [...state.movies, action.payload]
    return {
      ...state,
      movies: newMovies,
      showNotification: true,
      notificationContent: "Film byl přidán"
    }
  }

  if (action.type === "NO_MOVIE_NAME") {
    return {
      ...state,
      showNotification: true,
      notificationContent: "Zadejte film"
    }
  }

  return state
}
```

Pokud uživatel nezadá nic do políčka, tak vyskočí Zadejte film.



The image shows a web browser window with a purple header. The address bar displays 'localhost:30...'. The main content area has a white background with the text 'Zadejte film' in a serif font. Below this text is a text input field and a button labeled 'Přidat'.

Zadejte film

Přidat

47. React 2 - useReducer a notifikace - jak pracovat s chybou ve stavech

Video: <https://youtu.be/8tIQDqAsizQ>

App.js

Pokud se type nezachytí ani o "ADD_MOVIE", ani o "NO_MOVIE_NAME", tak je dobré nevracet stav, ale vyhodit chybu.

```
const reducer = (state, action) => {

  if (action.type === "ADD_MOVIE"){
    const newMovies = [...state.movies, action.payload]
    return {
      ...state,
      movies: newMovies,
      showNotification: true,
      notificationContent: "Film byl přidán"
    }
  }

  if (action.type === "NO_MOVIE_NAME"){
    return {
      ...state,
      showNotification: true,
      notificationContent: "Zadejte film"
    }
  }

  return new Error("Chyba - žádná shoda s action.type")
  // return state
}
```

48. React 2 - useReducer a notifikace - notifikace po 2 vteřinách zmizí

Video: <https://youtu.be/s1DPfzVo9Lc>

App.js

Vytvoříme funkci closeNotification, která zavolá dispatch pod názvem "CLOSE_NOTIFICATION".

Funkci pošleme do komponenty Modal.js pod libovolným názvem. Zde pod názvem closeNotif.

```
const closeNotification = () => {
  dispatch( {type: "CLOSE_NOTIFICATION"} )
}

return <section>
  {state.showNotification && <Modal
notifContent={state.notificationContent} closeNotif={closeNotification}
/>}
  <form onSubmit={submitForm}>
    <input type="text" onChange={ (e) => setMovieName(e.target.value)
}/>
    <input type="submit" value="Přidat" />
  </form>
```

App.js

V reducer nastavíme, co se má stát, když se spustí dispatch s tímto názvem.

```
if (action.type === "CLOSE_NOTIFICATION"){
  return {
    ...state,
    showNotification: false
  }
}

return new Error("Chyba - žádná shoda s action.type")
// return state
```

Modal.js

V Modal.js řekneme, kdy se má closeNotif spustit. My jsme zadali, že se má spustit po vypsání, takže použijeme useEffect a má ještě před spuštěním počkat 2 vteřiny.

```
import "./Modal.css"
import { useEffect } from "react"

const Modal = ({ notifContent, closeNotif } ) => {
```

```
useEffect( () => {
  setTimeout( () => {
    closeNotif()
  }, 2000)
})

return <div>
  <div>{notifContent}</div>
  { /* <p onClick={closeNotif}>X</p> */ }
</div>
}

export default Modal
```

49. React 2 - useReducer a notifikace - odstranění filmu ze seznamu

Video: <https://youtu.be/VyYEN6edrNs>

App.js

Přidáme tlačítko, na které když se klikne, tak to spustí v reduceru typ "REMOVE_MOVIE". A ještě posíláme informaci, na které ID bylo kliknuto.

```
<div>
  {state.movies.map( (oneMovie) => {
    return <div key={oneMovie.id}>
      <p>{oneMovie.name}</p>
      <button type="button" onClick={ () => dispatch( {type:
"REMOVE_MOVIE", payload: oneMovie.id} ) }>Smazat</button>
    </div>
  } )}
</div>
```

V reduceru převezmeme typ a vyfiltrujeme všechny filmy, které nemají dané ID. Ve filteredMovies zbudou pouze ty filmy, na které nebylo kliknuto. A podle toho upravíme stav.

```
if (action.type === "REMOVE_MOVIE"){
  const filteredMovies = state.movies.filter( (oneMovie) => {
    return oneMovie.id !== action.payload
  })

  return {
    ...state,
    movies: filteredMovies
  }
}

return new Error("Chyba - žádná shoda s action.type")
```

App.js - pročištění políčka po odeslání formuláře

Jako první nastavíme movieName na hodnotu prázdného stringu po odeslání.

```
const submitForm = (e) => {
  e.preventDefault()

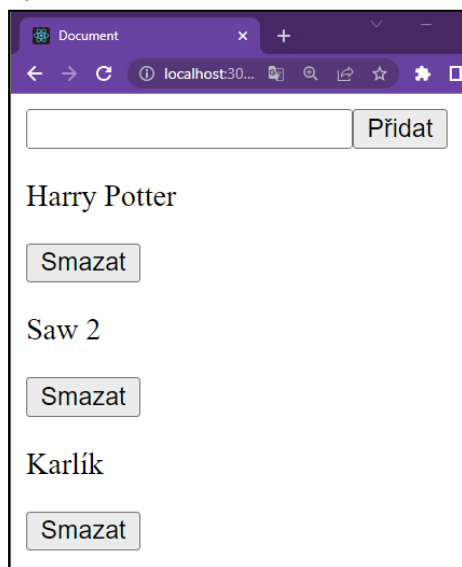
  if (movieName) {
    const newMovie = {id: new Date().getTime(), name: movieName}
    dispatch( {type: "ADD_MOVIE", payload: newMovie} )
  } else {
    dispatch( {type: "NO_MOVIE_NAME"} )
  }
}
```

```
    setMovieName("")  
  }  
}
```

U políčka nastavíme, že hodnota movieName má být aktuální hodnotou. Takže po odeslání se movieName nastaví na prázdný string = políčko se pročistí.

```
<form onSubmit={submitForm}>  
  <input type="text" value={movieName} onChange={ (e) =>  
setMovieName(e.target.value) }/>  
  <input type="submit" value="Přidat" />  
</form>
```

Výsledek na stránce



The screenshot shows a web browser window with a single tab titled 'Document'. The address bar shows 'localhost:30...'. The page content includes a text input field and a button labeled 'Přidat'. Below the input field, there is a list of three items, each with a 'Smazat' button next to it:

- Harry Potter
- Smazat
- Saw 2
- Smazat
- Karlík
- Smazat

50. React 2 - useReducer a notifikace - řešíme poskočení formuláře v CSS

Video: <https://youtu.be/rRQtAycTKCg>

Modal.js

```
return <div className="modal-box">
  <div className="modal-text">
    <p>{notifContent}</p>
  </div>
</div>
```

Modal.

```
.modal-box {
  position: relative;
}

.modal-text {
  position: absolute;
  top: 0px;
  left: 250px;
  font-size: 10px;
  background-color: #141414;
  color: #e5e5e5;
  width: 80px;
  height: 30px;
  border-radius: 50%;
  text-align: center;
  box-shadow: 0px 0px 10px #e50914;
}
```

51. React 2 - useReducer a notifikace - stylování

Video: https://youtu.be/u_eUqFMSgU

App.js - přidání className

```
return <section className="form">
  {state.showNotification && <Modal
notifContent={state.notificationContent} closeNotif={closeNotification}
/>}
  <form onSubmit={submitForm}>
    <input type="text" value={movieName} onChange={ (e) =>
setMovieName(e.target.value) }/>
    <input type="submit" value="Přidat" />
  </form>
  <div>
    {state.movies.map( (oneMovie) => {
      return <div key={oneMovie.id} className="all-movies">
        <p>{oneMovie.name}</p>
        <button type="button" onClick={ () => dispatch( {type:
"REMOVE_MOVIE", payload: oneMovie.id} ) }>Smazat</button>
      </div>
    } )}
  </div>
</section>
```

Index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  background-color: #141414;
  color: #e5e5e5;
}

.form {
  height: 100vh;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}

form {
```

```

        display: flex;
        flex-direction: column;
        align-items: center;
    }

    form input[type="text"]{
        margin-bottom: 10px;
        outline: none;
        border: 2px solid #e50914;
        padding: 5px 10px;
    }

    form input[type="submit"] {
        background-color: #e50914;
        border: none;
        padding: 5px 10px;
    }

    .all-movies {
        width: 300px;
        border: 2px solid #6a050a;
        display: flex;
        flex-direction: row;
        justify-content: space-between;
        padding: 5px 10px;
        margin-top: 10px;
    }

    .all-movies button {
        background-color: #e50914;
        border: none;
        padding: 3px 5px;
        font-size: 10px;
    }

```

Modal.js - přidání className

```

return <div>
    <div className="notification">{notifContent}</div>
</div>

```

Modal.css

```

.notification {

```

```
box-shadow: 0px 0px 10px #e50914;  
padding: 5px 10px;  
margin-bottom: 20px;  
}
```

Výsledek na stránce

Přidat

Harry Potter

Smazat

Saw 2

Smazat

Karlík a továrna na čokoládu

Smazat

52. React 2 - Prop drilling - 1. část

Video: <https://youtu.be/uF1VEqctIQs>

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result"></div>
</body>
</html>
```

index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

data.js

```
const movies = [
  { id: 1, name: 'Čung_e' },
  { id: 2, name: 'Zátah 2' },
  { id: 3, name: 'Zrození šampióna' },
  { id: 4, name: 'D-tox' },
]

export default movies
```

App.js

Naimportujeme data a pošleme je pod názvem data2 do komponenty MovieBox.js

```
import data from "../data"
import MovieBox from "../components/MovieBox"

const App = () => {
  return (
    <MovieBox data2={data}/>
  )
}

export default App
```

MovieBox.js

Zde přijdeme data pod názvem data2 a pošleme je pod názvem data3 do komponenty Movies.

```
import Movies from "../Movies"

const MovieBox = ({data2}) => {
  return (
    <Movies data3={data2}/>
  )
}

export default MovieBox
```

Movies.js

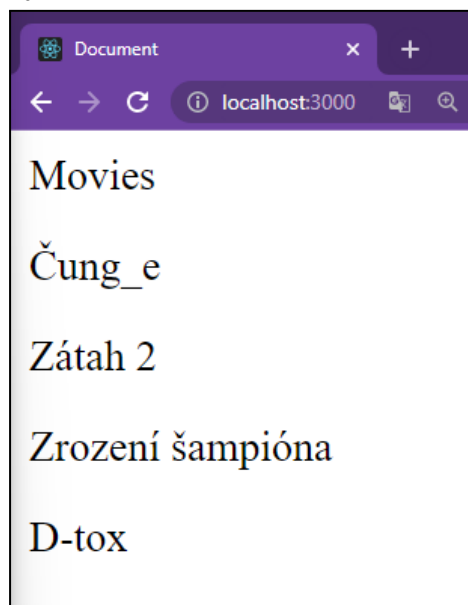
Zde přijmeme data pod názvem data3 a vypíšeme je pomocí map do stránky.

```
const Movies = ({data3}) => {
  return <div>
    <div>Movies</div>
    {/* {console.log(data3)} */}
    {data3.map( (oneMovie) => {
      const {id, name} = oneMovie

      return <div key={id}>
        <p>{name}</p>
      </div>
    })}
  </div>
}

export default Movies
```

Výsledek na stránce



53. React 2 - Prop drilling - 2. část

Video: <https://youtu.be/7GSgdtY6bdQ>

App.js

```
import data from "../data"
import MovieBox from "../components/MovieBox"
import { useState } from "react"

// Prop drilling
// useContext

const App = () => {
  const [movies, setMovies] = useState(data)

  const deleteMovie= (movieId) => {
    const newMovieList = movies.filter( (oneMovie) => {
      return oneMovie.id !== movieId
    })

    setMovies(newMovieList)
  }

  return (
    <MovieBox movies2={movies} deleteMovie={deleteMovie} />
  )
}

export default App
```

MovieBox.js

```
import Movies from "../Movies"

const MovieBox = ({movies2, deleteMovie}) => {
  return (
    <Movies movies3={movies2} deleteMovie={deleteMovie}/>
  )
}

export default MovieBox
```

Movies.js

```
const Movies = ({movies3, deleteMovie}) => {  
  return <div>  
  
    {movies3.map( (oneMovie) => {  
      const {id, name} = oneMovie  
  
      return <div key={id}>  
        <p>{name}</p>  
        <button onClick={ () =>  
deleteMovie(id) }>Smazat</button>  
      </div>  
    )}  
  
  </div>  
}  
  
export default Movies
```

Výsledek na stránce



54. React 2 - Nahrazujeme prop drilling pomocí useContext a createContext

Video: <https://youtu.be/O6R5ttoLzwc>

MoviesContext.js

Vytvoříme ve složce src nový soubor s názvem MoviesContext.js. Použijeme createContext, který začne načítat hodnotu value z vypsání komponenty, ve které bude použit název MoviesContext (ten jsme si sami zvolili). Vypsání komponenta bude v App.js.

```
import React from "react"

const MoviesContext = React.createContext()
export default MoviesContext
```

App.js

createContext si načte hodnotu value a my ji použijeme v Movies.js.

```
import data from "../data"
import MovieBox from "../components/MovieBox"
import { useState } from "react"
import MoviesContext from "../components/MoviesContext"

const App = () => {
  const [movies, setMovies] = useState(data)

  const deleteMovie= (movieId) => {
    const newMovieList = movies.filter( (oneMovie) => {
      return oneMovie.id !== movieId
    })

    setMovies(newMovieList)
  }

  return <MoviesContext.Provider value="ahoooooooooooooooooj">
    <MovieBox movies2={movies} deleteMovie={deleteMovie} />
  </MoviesContext.Provider>
}

export default App
```

Movies.js

Naimportujeme MoviesContext a také useContext. Pomocí useContext vytáhneme z MoviesContext hodnotu value, která existuje v pozadí a uložíme hodnotu do myData. V myData se nachází text "ahoooooooooooooooooj".

```
import MoviesContext from "../MoviesContext"
import { useContext } from "react"

const Movies = ({movies3, deleteMovie}) => {
  const myData = useContext(MoviesContext)
  console.log(myData)

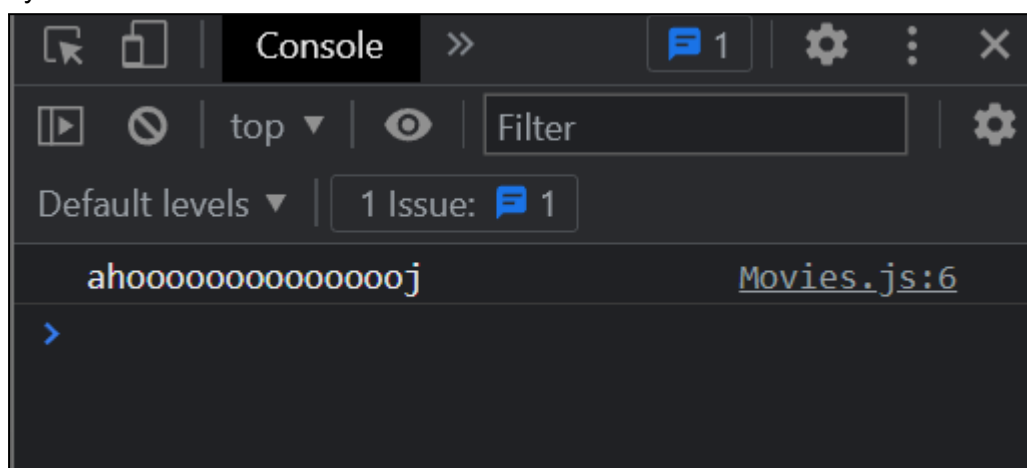
  return <div>
    {movies3.map( (oneMovie) => {
      const {id, name} = oneMovie

      return <div key={id}>
        <p>{name}</p>
        <button onClick={ () =>
deleteMovie(id) }>Smazat</button>
      </div>
    })}

  </div>
}

export default Movies
```

Výsledek v konzoli



55. React 2 - Upřesnění jak funguje useContext a createContext

Video: <https://youtu.be/lj3faR6esKs>

MovieBox.js

To, co jsme provedli v předchozím videu v App.js, tak provedeme i v MovieBox.js. Do value uložíme text “coooooooooooooooooo?”.

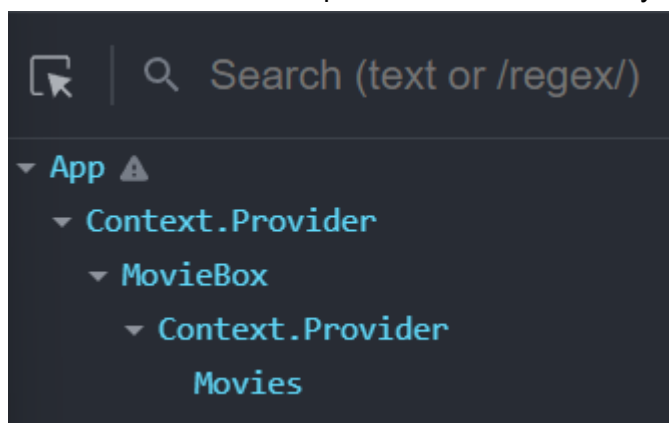
```
import Movies from "../Movies"
import MoviesContext from "../MoviesContext"

const MovieBox = ({movies2, deleteMovie}) => {

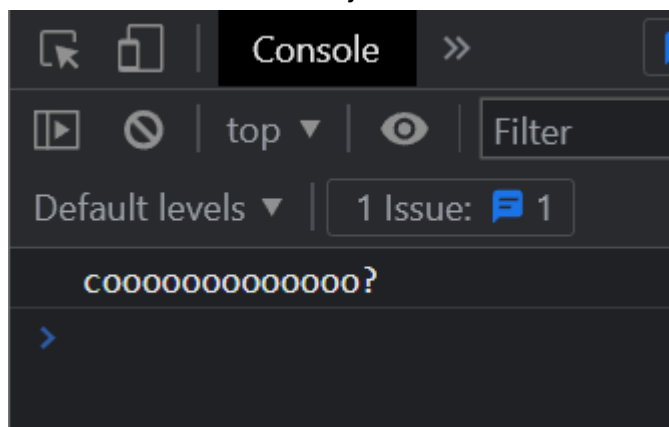
  return <MoviesContext.Provider value="coooooooooooooooooo?">
    <Movies movies3={movies2} deleteMovie={deleteMovie} />
  </MoviesContext.Provider>
}

export default MovieBox
```

V konzoli v záložce Components vidíme současný strom.



Protože v Movies.js se zavolá nejblíže Context.Provider a ten je hned nad Movies.js, tak Context.Provider začne hledat value v nejblíže nadřazeném souboru, který obsahuje název MoviesContext a najde ho v MovieBox a tam je ve value uložené “coooooooooooooooooo?”



56. React 2 - Pomocí useContext a createContext posíláme funkci

Video: <https://youtu.be/6D7AXjccXD8>

MovieBox.js

Tento soubor pročistíme, protože chceme value tahat z App.js a ne z MovieBox.js.

```
import Movies from "../Movies"

const MovieBox = ({movies2}) => {

  return (
    <Movies movies3={movies2} />
  )
}

export default MovieBox
```

App.js

Posíláme do MoviesContext.js pomocí providera hodnotu (value) jako funkci deleteMovie. I tuto červenou část můžete také smazat. Když ji tam ale necháme, tak se nic nestane.

```
return <MoviesContext.Provider value={ {deleteMovie} } >
  <MovieBox movies2={movies} deleteMovie={deleteMovie} />
</MoviesContext.Provider>
```

Movies.js

Vytáhli jsme si z MoviesContext pomocí useContext hodnotu value, což je deleteMovie a uložili jsme ji do proměnné deleteMovie. A tu již máme použitou níže u tlačítka.

```
import MoviesContext from "../MoviesContext"
import { useContext } from "react"

const Movies = ({movies3}) => {
  const { deleteMovie } = useContext(MoviesContext)

  return <div>
    {movies3.map( (oneMovie) => {
      const {id, name} = oneMovie

      return <div key={id}>
        <p>{name}</p>
        <button onClick={ () =>
deleteMovie(id) }>Smazat</button>
      </div>
    )}}
  </div>
}
```

```
</div>  
}  
  
export default Movies
```

57. React 2 - posíláme seznam filmů přes useContext a mažeme všechna props

Video: <https://youtu.be/DmpNTybIRA4>

App.js

Ve žluté části jsme odstranili všechny props a přidali jsme movies, které nyní z App.js posíláme do Movies.js

```
import data from "../data"
import MovieBox from "../components/MovieBox"
import { useState } from "react"
import MoviesContext from "../components/MoviesContext"

const App = () => {
  const [movies, setMovies] = useState(data)

  const deleteMovie= (movieId) => {
    const newMovieList = movies.filter( (oneMovie) => {
      return oneMovie.id !== movieId
    })

    setMovies(newMovieList)
  }

  return <MoviesContext.Provider value={ {deleteMovie, movies} }>
    <MovieBox />
  </MoviesContext.Provider>
}

export default App
```

MovieBox.js

Smazané všechny props, protože je již nepotřebujeme, když používáme useContext.

```
import Movies from "../Movies"

const MovieBox = () => {

  return (
    <Movies />
  )
}

export default MovieBox
```

Movies.js

Opět smazané všechna props a vypisujeme vše do stránky z proměnné movies, kterou jsme si poslali z App.js.

```
import MoviesContext from "../MoviesContext"
import { useContext } from "react"

const Movies = ({}) => {

  const { deleteMovie, movies } = useContext(MoviesContext)

  return <div>
    {movies.map( (oneMovie) => {
      const {id, name} = oneMovie

      return <div key={id}>
        <p>{name}</p>
        <button onClick={ () =>
deleteMovie(id) }>Smazat</button>
      </div>
    })}

  </div>
}

export default Movies
```

MoviesContext.js

Tento soubor zůstává stále stejný.

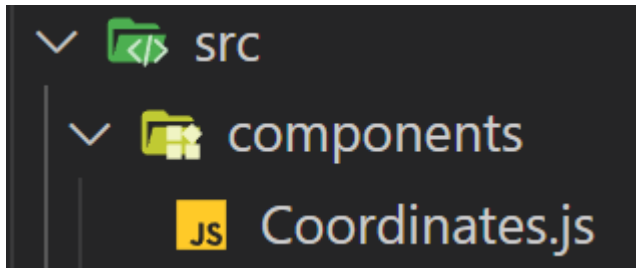
```
import React from "react"

const MoviesContext = React.createContext()
export default MoviesContext
```

58. React 2 - vlastní hook a loading - vytvoření loadingu

Video: <https://youtu.be/F2gcS3e-sq0>

Ve složce src vytvoříme složku components a uvnitř složky components založíme soubor Coordinates.js



Coordinates.js

Vytvoříme componentu a připravíme si vše pro loading. Pokud je loading true, tak se zobrazí text "loading...". Pokud bychom loading nastavili na false, tak se vypíše do stránky test "data".

```
import { useState } from "react"

const Coordinates = () => {
  const [loading, setLoading] = useState(true)

  return <div>
    <div>{loading ? "loading..." : "data"}</div>
  </div>
}

export default Coordinates
```

App.js

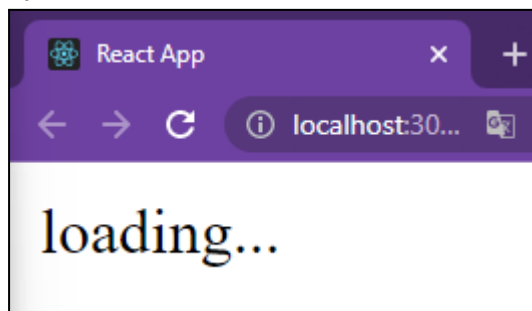
Coordinates naimportujeme do App.js a vypíšeme do stránky.

```
import Coordinates from "../components/Coordinates"

const App = () => {
  return <div>
    <Coordinates />
  </div>
}

export default App
```

Výsledek na stránce



59. React 2 - vlastní hook a loading - načítáme data z API

Video: <https://youtu.be/A0A5DlrmDbs>

International space station API:

<http://open-notify.org/Open-Notify-API/ISS-Location-Now/>

data jsou na této adrese: <http://api.open-notify.org/iss-now.json>

Coordinates.js

Vytvoříme proměnnou coordinates, do které uložíme zatím prázdný objekt. Dále napíšeme funkci, která bude načítat data a uloží je do proměnné issCoordinates. Obsah proměnné issCoordinates uložíme pomocí setCoordinates do proměnné coordinates. Když máme načtená data, tak také nastavíme proměnnou loading pomocí setLoading na false. Funkci pro načítání dat getCoordinates zavoláme až na konci, takže pomocí useEffect.

```
import { useState, useEffect } from "react"

const url = "http://api.open-notify.org/iss-now.json"

const Coordinates = () => {
  const [loading, setLoading] = useState(true)
  const [coordinates, setCoordinates] = useState({})

  const getCoordinates = async () => {
    const response = await fetch(url)
    const issCoordinates = await response.json()
    setLoading(false)
    setCoordinates(issCoordinates)
  }

  useEffect( () => {
    getCoordinates()
  }, [])

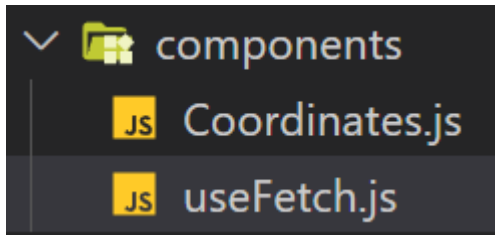
  return <div>
    <div>{loading ? "loading..." : "data"}</div>
  </div>
}

export default Coordinates
```

60. React 2 - vlastní hook a loading - vytváříme vlastní hook

Video: <https://youtu.be/jlATUbjTfw>

Ve složce components založíme soubor useFetch.js



useFetch.js

Žlutou barvou je to, co jsme zkopírovali z Coordinates.js a zeleně to, co jsme dopsali.

```
import { useState, useEffect } from "react"

const useFetch = (url) => {
  const [loading, setLoading] = useState(true)
  const [coordinates, setCoordinates] = useState({})

  const getCoordinates = async () => {
    const response = await fetch(url)
    const issCoordinates = await response.json()
    setLoading(false)
    setCoordinates(issCoordinates)
  }

  useEffect(() => {
    getCoordinates()
  }, [])

  return {coordinates, loading}
}

export default useFetch
```

Coordinates.js

Naimportovali jsme useFetch a vyexportované coordinates a loading jsme si uložili do stejnojmenných proměnných. Do useFetch jsme poslali proměnnou apiUrl.

```
import { useState, useEffect } from "react"
import useFetch from "../useFetch"

const apiUrl = "http://api.open-notify.org/iss-now.json"

const Coordinates = () => {
  const {coordinates, loading} = useFetch(apiUrl)

  return <div>
    <div>{loading ? "loading..." : "data"}</div>
  </div>
}

export default Coordinates
```

61. React 2 - vlastní hook a loading - vypisujeme data a upravujeme kód

Video: https://youtu.be/Bgt3t_6uZyE

useFetch.js

Necháme useEffect spustit až ve chvíli, kdy se změní URL.

Pro lepší vypovídající schopnost coordinates přejmenujeme na data.

```
import { useState, useEffect } from "react"

const useFetch = (url) => {
  const [loading, setLoading] = useState(true)
  const [data, setData] = useState({})

  const getCoordinates = async () => {
    const response = await fetch(url)
    const issCoordinates = await response.json()
    setLoading(false)
    setData(issCoordinates)
  }

  useEffect(() => {
    getCoordinates()
  }, [url])

  return { data, loading }
}

export default useFetch
```

Coordinates.js

I zde musíme přejmenovat coordinats na data.

Poté ještě vypíšeme z data potřebné informace do stránky.

```
import useFetch from "../useFetch"

const apiUrl = "http://api.open-notify.org/iss-now.json"

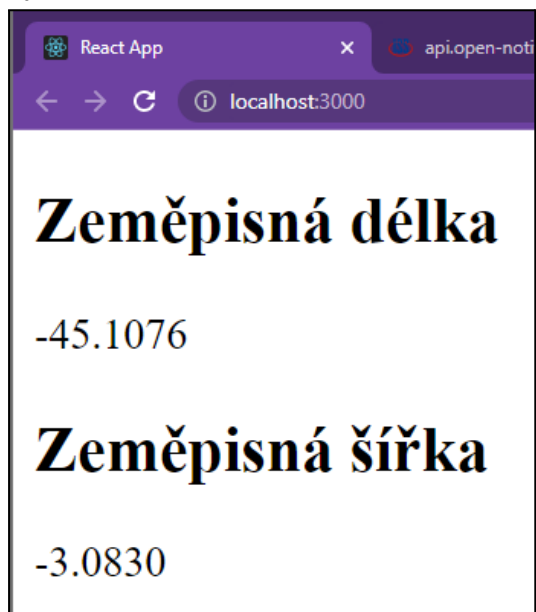
const Coordinates = () => {
  const { data, loading } = useFetch(apiUrl)

  return <div>
    <div>{loading ? "loading..." : <div>
      <h2>Zeměpisná délka</h2>
```

```
        <p>{data.iss_position.longitude}</p>
        <h2>Zeměpisná šířka</h2>
        <p>{data.iss_position.latitude}</p>
      </div>
    }
  </div>
</div>
}

export default Coordinates
```

Výsledek na stránce



62. React 2 - vlastní hook a loading - odstraňujeme varování

Video: <https://youtu.be/u4Oha8xV4As>

useFetch.js

Pokud se chcete zbavit varování v terminálu, tak funkci `getCoordinates` přesuňte dovnitř do `useEffect`.

```
import { useState, useEffect } from "react"

const useFetch = (url) => {
  const [loading, setLoading] = useState(true)
  const [data, setData] = useState({})

  useEffect(() => {
    const getCoordinates = async () => {
      const response = await fetch(url)
      const issCoordinates = await response.json()
      setLoading(false)
      setData(issCoordinates)
    }
    getCoordinates()
  }, [url])

  return { data, loading }
}

export default useFetch
```

63. React 2 - validujeme props - nastavení

Video: <https://youtu.be/ziHYKXAi6QQ>

data.js

```
const books = [
  {
    id: 1,
    image:
      "https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-kamen-mudrcu-9788000061917.jpg.webp",
    title: "Harry Potter a Kámen mudrců",
    price: 890
  },
  {
    id: 2,
    image:
      "https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/333420609/10.jpg.webp",
    title: "Harry Potter a Tajemná komnata",
    price: 690
  },
  {
    id: 3,
    image:
      "https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-vezen-z-azkabanu-9788000063393.jpg.webp",
    title: "Harry Potter a Vězeň z Azkabanu",
    price: 990
  },
]

export default books
```

App.js

```
import Books from "../components/Books"

const App = () => {
  return (
    <Books />
  )
}

export default App
```

Books.js

Data vypíšeme pomocí map, ale použijeme k tomu komponentu. Musíme data do komponenty poslat, abychom mohli vše nastavit v Book.js

```
import Book from "../Book"
import data from "../data"

const Books = () => {
  return <div>
    {data.map( (oneBook) => {
      return <Book key={oneBook.id} {...oneBook} />
    })}
  </div>
}

export default Books
```

Book.js

```
const Book = ({image, title, price}) => {
  return <div>
    <h2>{title}</h2>
    <img src={image} alt="" />
    <p>{price} Kč</p>
  </div>
}

export default Book
```

Výsledek na stránce

Harry Potter a Kámen mudrců



890 Kč

Harry Potter a Tajemná komnata



690 Kč

64. React 2 - validujeme props - zjišťujeme chybějící data v knížkách (propTypes)

Video: <https://youtu.be/WcQ9SZuOOo4>

Book.js

Naimportujeme PropTypes. Níže provedeme zápis, který bude kontrolovat, jestli máme všechny obrázky, titly a ceny. Vždy musíme doplnit správný datový typ.

```
import PropTypes from "prop-types"

const Book = ({image, title, price}) => {
  return <div>
    <h2>{title}</h2>
    <img src={image} alt="" />
    <p>{price} Kč</p>
  </div>
}

Book.propTypes = {
  image:PropTypes.string.isRequired,
  title:PropTypes.string.isRequired,
  price:PropTypes.number.isRequired,
}

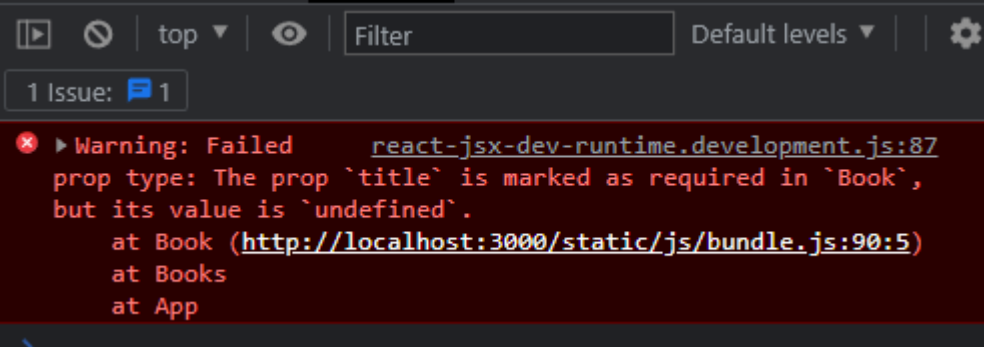
export default Book
```

data.js

Pokud si nyní např. u 3. knihy zakomentujete title (bude chybět), tak se v konzoli objeví hlášky, že title chybí.

```
{
  id: 3,
  image:
"https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-vezen-z-azkabanu-9788000063393.jpg.webp",
  // title: "Harry Potter a Vězeň z Azkabanu",
  price: 990
},
```

Výsledek v konzoli



The screenshot shows a web browser's developer console with a dark theme. At the top, there is a toolbar with icons for opening the console, disabling it, and a 'top' dropdown. To the right of the toolbar is a 'Filter' input field and a 'Default levels' dropdown. Below the toolbar, a summary bar indicates '1 Issue: 1'. The main area of the console displays a warning message in red text: 'Warning: Failed prop type: The prop `title` is marked as required in `Book`, but its value is `undefined`.' The message is followed by the source file 'react-jsx-dev-runtime.development.js:87' and a stack trace with three entries: 'at Book (http://localhost:3000/static/js/bundle.js:90:5)', 'at Books', and 'at App'.

```
⌵ 1 Issue: 1
Warning: Failed prop type: The prop `title` is marked as required in `Book`,
but its value is `undefined`.
    at Book (http://localhost:3000/static/js/bundle.js:90:5)
    at Books
    at App
```

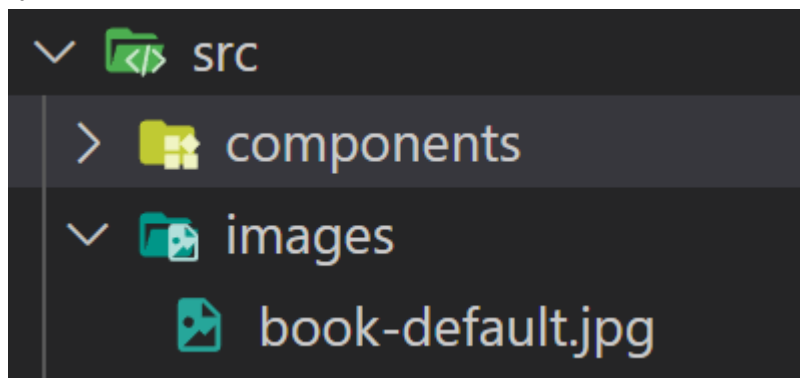
65. React 2 - validujeme props - výchozí hodnoty pomocí defaultProps

Video: <https://youtu.be/6f7KOjxz4VQ>

Na tomto odkazu najdete obrázek:

https://drive.google.com/drive/folders/1Dx9A-9wZN5QTk2vbbJt3f8NxuG_NMI4J?usp=sharing

Ve složce src založíme složku images a do ní stáhneme obrázek, který najdete na odkazu výše.



Book.js

Naimportujeme obrázek ze složky images.

Níže zadáváme defaultní (výchozí) hodnoty.

```
import PropTypes from "prop-types"
import defaultImage from "../images/book-default.jpg"

const Book = ({image, title, price}) => {
  return <div>
    <h2>{title}</h2>
    <img src={image} alt="" />
    <p>{price} Kč</p>
  </div>
}

Book.propTypes = {
  image:PropTypes.string.isRequired,
  title:PropTypes.string.isRequired,
  price:PropTypes.number.isRequired,
}

Book.defaultProps = {
  image:defaultImage,
  title:"Defaultní název",
  price:1000,
```

```
}  
  
export default Book
```

data.js

Zakomentujeme informace u 3. knihy v data.js

```
{  
  id: 3,  
  // image:  
  "https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-vezen-z-azkabanu-9788000063393.jpg.webp",  
  // title: "Harry Potter a Vězeň z Azkabanu",  
  // price: 990  
},
```

Výsledek na stránce - zobrazí se místo 3. knihy defaultní hodnoty



66. React 2 - validujeme props - další způsob zápisu

Video: <https://youtu.be/4RQ3mJgYolA>

Book.js

Máme ještě další způsob zápisu defaultních hodnot. Můžeme je zapisovat pomocí logického operátoru nebo, který se píše jako dvě svislítka || (pravý Alt + w)

```
import PropTypes from "prop-types"
import defaultImage from "../images/book-default.jpg"

const Book = ({image, title, price}) => {
  return <div>
    <h2>{title || "Defaultní název"}</h2>
    <img src={image || defaultImage} alt="" />
    <p>{price || 1000} Kč</p>
  </div>
}

Book.propTypes = {
  image:PropTypes.string.isRequired,
  title:PropTypes.string.isRequired,
  price:PropTypes.number.isRequired,
}

// Book.defaultProps = {
//   image:defaultImage,
//   title:"Defaultní název",
//   price:1000,
// }

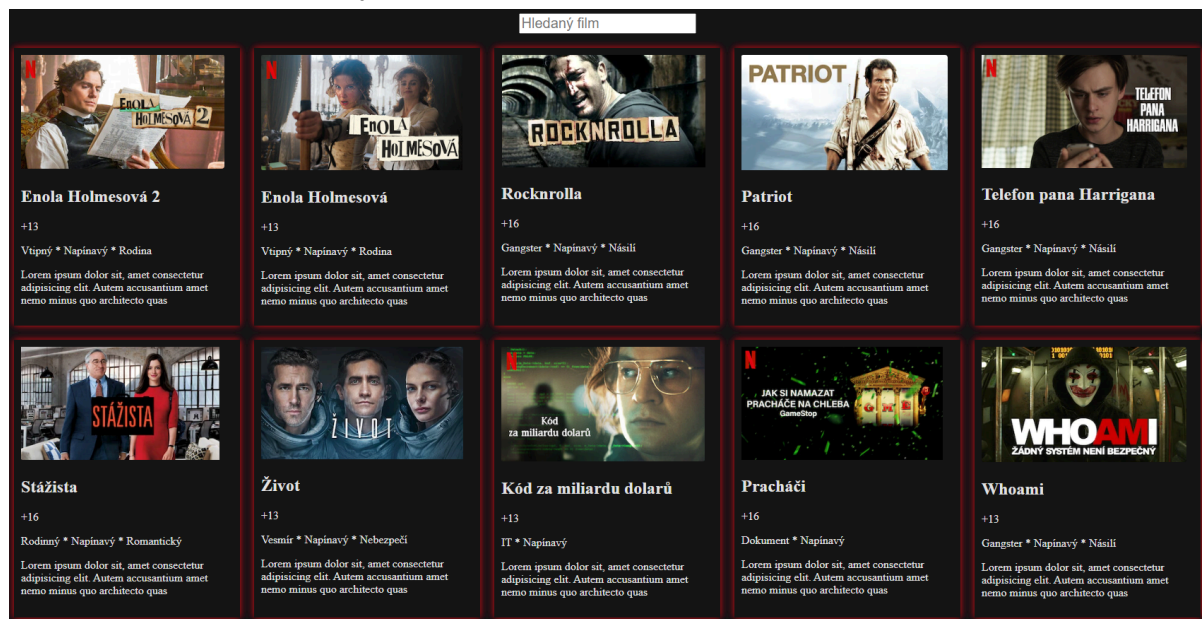
export default Book
```

Zkratka pro doplnění componenty včetně propTypes je: rafcp

67. React 2 - Vyhledávací políčko - ukázka

Video: <https://youtu.be/BBkiDFS4HCK>

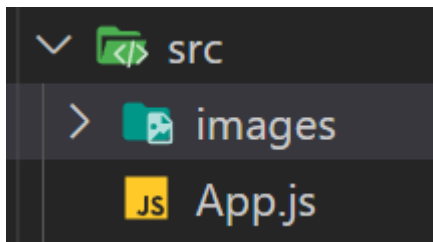
Takto bude naše aplikace vypadat.



68. React 2 - Vyhledávací políčko - nastavení projektu

Video: <https://youtu.be/zHFKZcJd91Y>

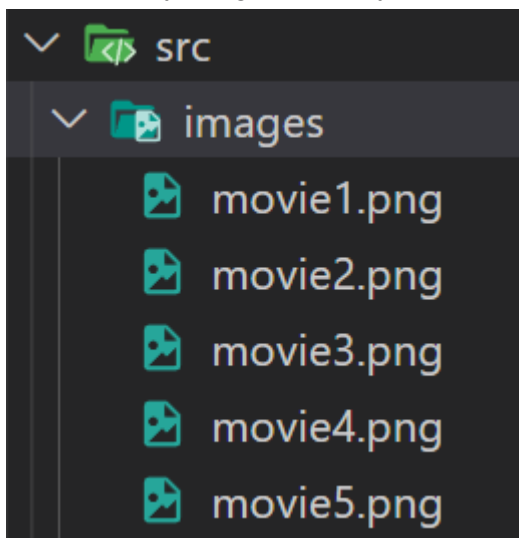
Ve složce src vytvoříme složku images



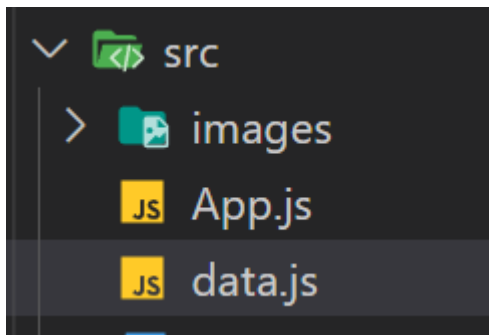
Do ní vložíme obrázky z tohoto odkazu

<https://drive.google.com/drive/folders/1td236jZt7gvk1WxD2YzB9Edot7AxMk2P?usp=sharing>

Obsah složky images bude vypadat takto



Ve složce src vytvoříme soubor data.js.



Do něj vložíme tato data včetně naimportovaných obrázků.

```
import image1 from "../images/movie1.png"
import image2 from "../images/movie2.png"
import image3 from "../images/movie3.png"
import image4 from "../images/movie4.png"
import image5 from "../images/movie5.png"
import image6 from "../images/movie6.png"
```

```
import image7 from "./images/movie7.png"
import image8 from "./images/movie8.png"
import image9 from "./images/movie9.png"
import image10 from "./images/movie10.png"

const allMovies = [
  {
    id: 1,
    image: image1,
    title: "Enola Holmesová 2",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
  },
  {
    id: 2,
    image: image2,
    title: "Enola Holmesová",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
  },
  {
    id: 3,
    image: image3,
    title: "Rocknrolla",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
  },
  {
    id: 4,
    image: image4,
    title: "Patriot",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
  },
  {
    id: 5,
    image: image5,
    title: "Telefon pana Harrigana",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
  },
  {
    id: 6,
    image: image6,
    title: "Stážista",
    age: "+16",
    tags: "Rodinný * Napínavý * Romantický",
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
  },
]
```

```

    {
      id: 7,
      image: image7,
      title: "Život",
      age: "+13",
      tags: "Vesmír * Napínavý * Nebezpečí",
      description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
    },
    {
      id: 8,
      image: image8,
      title: "Kód za miliardu dolarů",
      age: "+13",
      tags: "IT * Napínavý",
      description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
    },
    {
      id: 9,
      image: image9,
      title: "Pracháči",
      age: "+16",
      tags: "Dokument * Napínavý",
      description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
    },
    {
      id: 10,
      image: image10,
      title: "Whoami",
      age: "+13",
      tags: "Gangster * Napínavý * Násilí",
      description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium
amet nemo minus quo architecto quas",
    },
  ],

export default allMovies

```

App.js

Naimportujeme useState a useEffect. Také naimportujeme data z data.js.

```

import { useState, useEffect } from "react"
import allMovies from "../data"

const App = () => {
  return (
    <div>App</div>
  )
}

export default App

```

69. React 2 - Vyhledávací políčko - tvorba políčka a vypsání filmů

Video: <https://youtu.be/QNyk338BxnY>

App.js

Vytvoříme formulář (žlutá část kódu), kde pomocí onChange zachytáváme, co se píše do políčka a ukládáme si to do proměnné searchingText. Níže vypisujeme do stránky všechny filmy (zelená část kódu) uložené v allMovies.

```
import { useState, useEffect } from "react"
import allMovies from "../data"

const App = () => {
  const [searchingText, setSearchingText] = useState("")

  return <div>
    <form>
      <input type="text" placeholder="Hledaný film" onChange={
        (e) => setSearchingText(e.target.value)
      }/>
    </form>
    <div>
      {allMovies.map( (oneMovie) => {
        const {id, image, title, age, tags, description} = oneMovie

        return <div key={id}>
          <img src={image} alt="" />
          <h2>{title}</h2>
          <p>{age}</p>
          <p>{tags}</p>
          <p>{description}</p>
        </div>
      ) )}
    </div>
  </div>
}

export default App
```

Výsledek na stránce

Hledaný film



Enola Holmesová 2

+13

Vtipný * Napínavý * Rodina

Lorem ipsum dolor sit, amet consectetur
adipiscing elit. Autem accusantium amet nemo
minus quo architecto quas



70. React 2 - Vyhledávací políčko - zprovozňujeme vyhledávání

Video: <https://youtu.be/G4krTgtGhUI>

App.js

Pomocí useEffectu profiltrujeme všechny filmy v allMovies a ty, kde název odpovídá hledanému textu, tak vrátíme do moviesAfterFilter. Pomocí setFilteredMovies nastavíme MoviesAfterFilter do proměnné filteredMovies a ty vypíšeme dole do stránky.

```
import { useState, useEffect } from "react"
import allMovies from "../data"

const App = () => {
  const [searchingText, setSearchingText] = useState("")
  const [filteredMovies, setFilteredMovies] = useState([])

  useEffect( () => {
    const moviesAfterFilter = allMovies.filter( (oneMovie) => {
      return
oneMovie.title.toLowerCase().includes(searchingText.toLowerCase())
    })
    setFilteredMovies(moviesAfterFilter)
  }, [searchingText])

  return <div>
    <form>
      <input type="text" placeholder="Hledaný film" onChange={
        (e) => setSearchingText(e.target.value)
      }/>
    </form>
    <div>
      {filteredMovies.map( (oneMovie) => {
        const {id, image, title, age, tags, description} = oneMovie

        return <div key={id}>
          <img src={image} alt="" />
          <h2>{title}</h2>
          <p>{age}</p>
          <p>{tags}</p>
          <p>{description}</p>
        </div>
      ) }}
    </div>
  </div>
```

```
}  
  
export default App
```

71. React 2 - Vyhledávací políčko - stylujeme aplikaci

Video: https://youtu.be/1j44U_DkFH0

App.js

Přidáme classy.

```
import { useState, useEffect } from "react"
import allMovies from "../data"

const App = () => {
  const [searchingText, setSearchingText] = useState("")
  const [filteredMovies, setFilteredMovies] = useState([])

  useEffect( () => {
    const moviesAfterFilter = allMovies.filter( (oneMovie) => {
      return
oneMovie.title.toLowerCase().includes(searchingText.toLowerCase())
    })
    setFilteredMovies(moviesAfterFilter)
  }, [searchingText])

  return <div className="movies-box">
    <form>
      <input type="text" placeholder="Hledaný film" onChange={
        (e) => setSearchingText(e.target.value)
      }/>
    </form>
    <div className="all-movies">
      {filteredMovies.map( (oneMovie) => {
        const {id, image, title, age, tags, description} = oneMovie

        return <div key={id} className="one-movie">
          <img src={image} alt="" />
          <h2>{title}</h2>
          <p>{age}</p>
          <p>{tags}</p>
          <p>{description}</p>
        </div>
      ) }}
    </div>
  </div>
}

export default App
```

Index.css

```
body {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  background-color: #141414;
  color: #e5e5e5;
}

.movies-box {
  display: flex;
  flex-direction: column;
  align-items: center;
}

.movies-box input {
  margin: 10px;
  outline: none;
  font-size: 20px;
}

.all-movies {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
  justify-content: center;
}

.one-movie {
  width: 300px;
  box-shadow: 0px 0px 10px #e50914;
  margin: 10px;
  padding: 10px;
}

.one-movie img {
  width: 100%;
}
```

Hledaný film



Enola Holmesová 2

+13

Vtipný * Napínavý * Rodina

Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet
nemo minus quo architecto quas



Enola Holmesová

+13

Vtipný * Napínavý * Rodina

Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet
nemo minus quo architecto quas



Rocknrolla

+16

Gangster * Napínavý * Násilí

Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet
nemo minus quo architecto quas



Patriot

+16

Gangster * Napínavý * Násilí

Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet
nemo minus quo architecto quas

72. React 2 - Závěrečné video ke kurzu React 2

Video: https://youtu.be/nlQ3UnjGt_A

Uvidíme se u kurzu React 3...